

Significance Tests

T-Tests and Chi-Square

1. Why Statistical Significance Tests?

In the **data science life cycle**, statistical significance tests are integrated across almost every phase to ensure that decisions are based on **reliable evidence** rather than random noise or chance.

2. Core Concepts & Important Terms

Null & Alternative Hypotheses

Term	Definition
Null hypothesis (H_0)	<i>Nothing has changed.</i> There is no difference, no effect, no relationship.
Alternative hypothesis (H_1)	<i>Something has changed.</i> There is a difference, an effect, or a relationship.

Intuition: Think of H_0 as "innocent until proven guilty." We start by assuming nothing interesting is happening. We only conclude something *is* happening if the data provides strong enough evidence.

p-value

The **p-value** tells you how likely it is to observe your results (or more extreme results) *if the null hypothesis were true*.

- **Small p-value** → the result is unlikely to be random → evidence against H_0 .
- **Large p-value** → the result could easily occur by chance → no strong evidence against H_0 .

Intuition: Imagine flipping a coin 10 times and getting 10 heads. If the coin were fair (H_0), this is very unlikely ($p = (1/2)^{10} = 1/1024 \approx 0.001$). That tiny probability is the p-value. It's NOT the probability that H_0 is true — it's the probability of seeing your data *if* H_0 were true.

Common p-value misconceptions:

- ❌ " $p = 0.04$ means there's a 4% chance the null hypothesis is true." ✅ Reality: p-value is the probability of observing data at least as extreme as ours, **given** H_0 is true.
- ❌ " $p = 0.04$ is way better than $p = 0.06$." ✅ Reality: The 0.05 threshold is **arbitrary**. Both suggest weak-to-moderate evidence. Don't treat 0.05 as a sharp dividing line.
- ❌ "Non-significant results mean no effect." ✅ Reality: Failure to reject H_0 doesn't prove H_0 is true. Maybe the sample size was too small to detect the effect.

Significance Level (α)

α (alpha) is your **threshold** for deciding whether a p-value is "small enough" to reject H_0 .

- Most commonly: $\alpha = 0.05$ (5%)
- If $p\text{-value} \leq \alpha \rightarrow$ **Reject H_0** (statistically significant)
- If $p\text{-value} > \alpha \rightarrow$ **Fail to reject H_0** (not statistically significant)

Intuition: α is the maximum probability of making a Type I Error (false alarm) you are willing to accept. Setting $\alpha = 0.05$ means you accept a 5% risk of wrongly rejecting a true null hypothesis.

Sample Mean vs Population Mean

Term	Symbol	Meaning
Population mean (true mean)	μ (mu)	The precise average of all data points in the entire population
Sample mean	$\bar{x}$ (x-bar)	The average of a smaller group (sample) taken from the population — an estimate of μ

Since we rarely have access to the entire population, we work with samples and use the sample mean to estimate the population mean. This is why all hypothesis tests involve uncertainty.

3. Confidence Intervals

What is a Confidence Interval?

A **confidence interval (CI)** provides a **range of values** that likely contains the true population parameter (e.g., the true mean). It gives you more depth than a p-value alone by showing the **uncertainty** around your estimate.

A conversion rate improvement of **1.2%** with a **95% CI of [0.8%, 1.6%]** is far more useful than a simple p-value — it tells you both the direction and the plausible range of the true effect.

Formula for the Confidence Interval (using t-distribution):

$$CI = \bar{x} \pm t_{\text{critical}} \times SE$$

Where:

- $\bar{x}$ = sample mean
- t_{critical} = critical value from the t-distribution (depends on confidence level and df)
- $SE = \text{Standard Error} = s / \sqrt{n}$
- s = sample standard deviation
- n = sample size
- df = degrees of freedom = $n - 1$

Intuition Behind Confidence Intervals

Important: A 95% CI does **not** mean "there is a 95% probability that the true mean lies in this specific interval." Rather, it means: if we repeated the sampling procedure many times and calculated a CI each time, **95% of those intervals would contain the true mean.**

The sample mean $\bar{x}$ will differ from the true mean μ . The CI acknowledges this uncertainty and gives a plausible range for μ .

Example 1: Customer Satisfaction Score CI

A hotel wants to estimate the average satisfaction score of all 500 guests (the population). They survey 150 guests (the sample) and compute a 95% confidence interval for the true mean satisfaction score.

Steps:

1. Generate the population (500 guests).
2. Draw a random sample of 150 guests.
3. Compute the sample mean and standard deviation.
4. Use the t-distribution to find the critical value.
5. Compute the margin of error and the CI.

```
# — Step 1: Generate the full population (500 guests) —————
np.random.seed(42)
population_size = 500
true_mean = 7.8 # Average satisfaction
true_std = 1.5 # Variation spread
population = np.random.normal(loc=true_mean, scale=true_std, size=population_size)
```

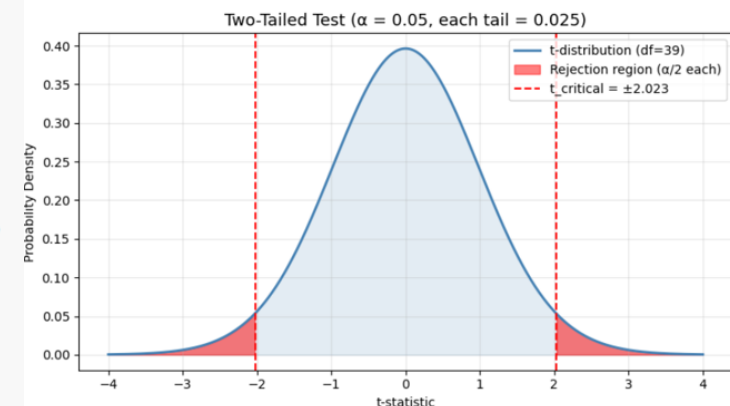
```
# — Step 2: Draw a sample (150 guests) —————
sample_size = 150
sample = np.random.choice(population, size=sample_size, replace=False)
# replace=False:
# Once an element is picked, it cannot be picked again (sampling without replacement).
```

```
# — Step 3: Calculate sample statistics —————
sample_mean = np.mean(sample)
sample_std = np.std(sample, ddof=1) # ddof=1 → unbiased (Bessel's correction)
# ddof=1 divide by n-1 instead of n, which gives an unbiased estimate of the population standard deviation.
confidence_level = 0.95
```

```
# — Step 4: Confidence interval calculation —————
se = sample_std / np.sqrt(sample_size) # standard Error
df = sample_size - 1
# ppf: percent-point function (inverse CDF).
# (1 + confidence_level) / 2 = 0.975 captures the middle 95% (two-tailed).
t_critical = stats.t.ppf((1 + confidence_level) / 2, df)
```

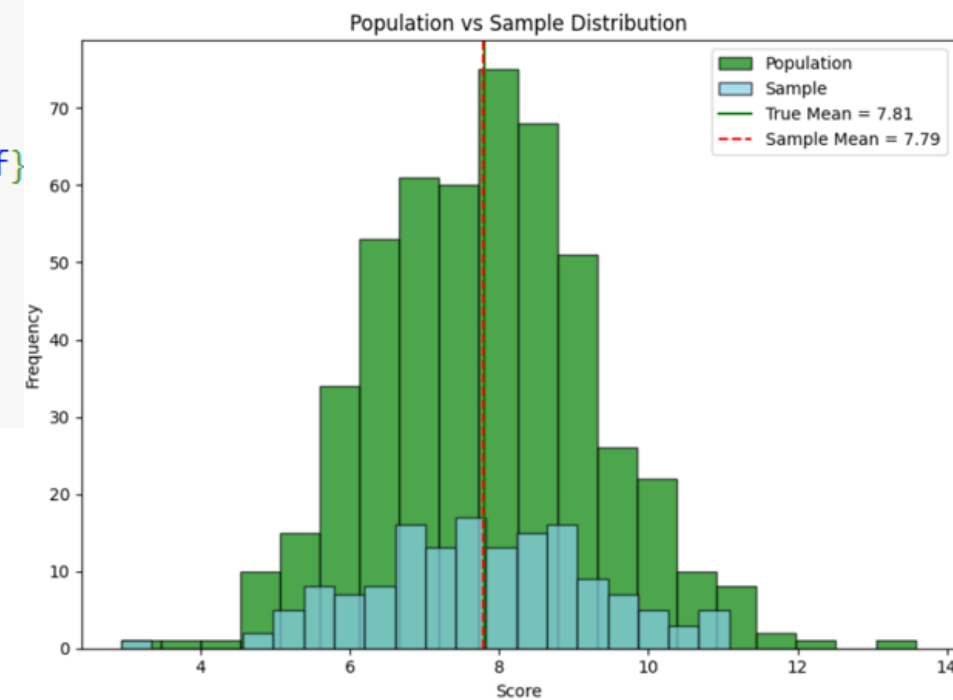
```
margin_of_error = t_critical * se
ci_lower = sample_mean - margin_of_error
ci_upper = sample_mean + margin_of_error
```

- `stats.t.ppf(q, df)` – **Percent Point Function** (inverse CDF) of the t-distribution. Returns the t-value such that `q` fraction of the distribution lies to its left. We pass `0.975` ($= (1 + 0.95)/2$) to capture the middle 95% for a two-tailed interval.




```
# — Step 6: Visualization —
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.patch.set_facecolor('white')

# — Left: Population vs Sample histogram —
axes[0].set_facecolor('white')
axes[0].hist(population, bins=20, alpha=0.7, label="Population", color="green", edgecolor="black",
)
axes[0].hist(
    sample, bins=20, alpha=0.7, label="Sample", color="skyblue", edgecolor="black"
)
axes[0].axvline(
    np.mean(population),
    color="green",
    label=f"True Mean = {np.mean(population):.2f}",
)
axes[0].axvline(
    sample_mean, color="red", linestyle="--", label=f"Sample Mean = {sample_mean:.2f}"
)
axes[0].legend()
axes[0].set_title("Population vs Sample Distribution")
axes[0].set_xlabel("Score")
axes[0].set_ylabel("Frequency")
```



```
margin_of_error = t_critical * se
```

```
# — Right: Confidence interval visualization —
```

```
axes[1].set_facecolor('white')
```

```
axes[1].errorbar(
```

```
    x=1,
```

```
    y=sample_mean,
```

```
    yerr=margin_of_error,
```

```
    fmt='o',
```

```
    color='blue',
```

```
    ecolor='black',
```

```
    capsize=10,
```

```
    label=f"Sample Mean  $\pm$  Margin of Error",
```

```
)
```

```
axes[1].axhline(np.mean(population), color='green', linestyle='-', label="True Mean")
```

```
axes[1].set_xlim(0, 2)
```

```
axes[1].set_xticks([])
```

```
axes[1].set_title("95% Confidence Interval")
```

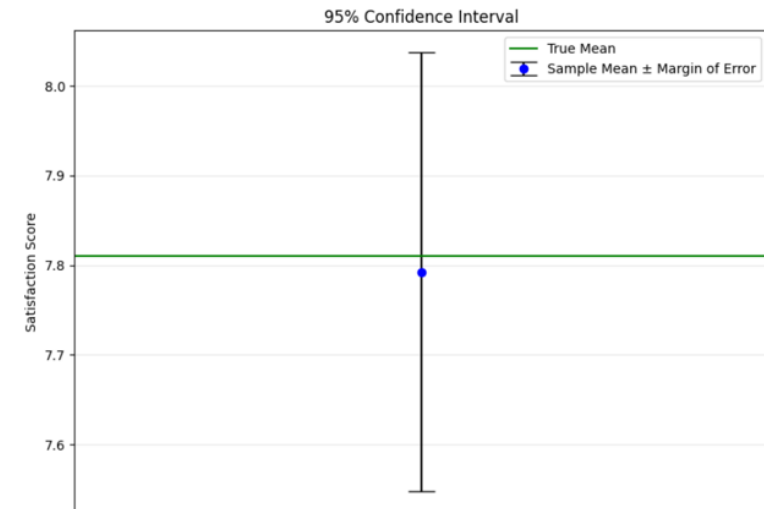
```
axes[1].set_ylabel("Satisfaction Score")
```

```
axes[1].legend()
```

```
axes[1].grid(True, alpha=0.3)
```

```
plt.tight_layout()
```

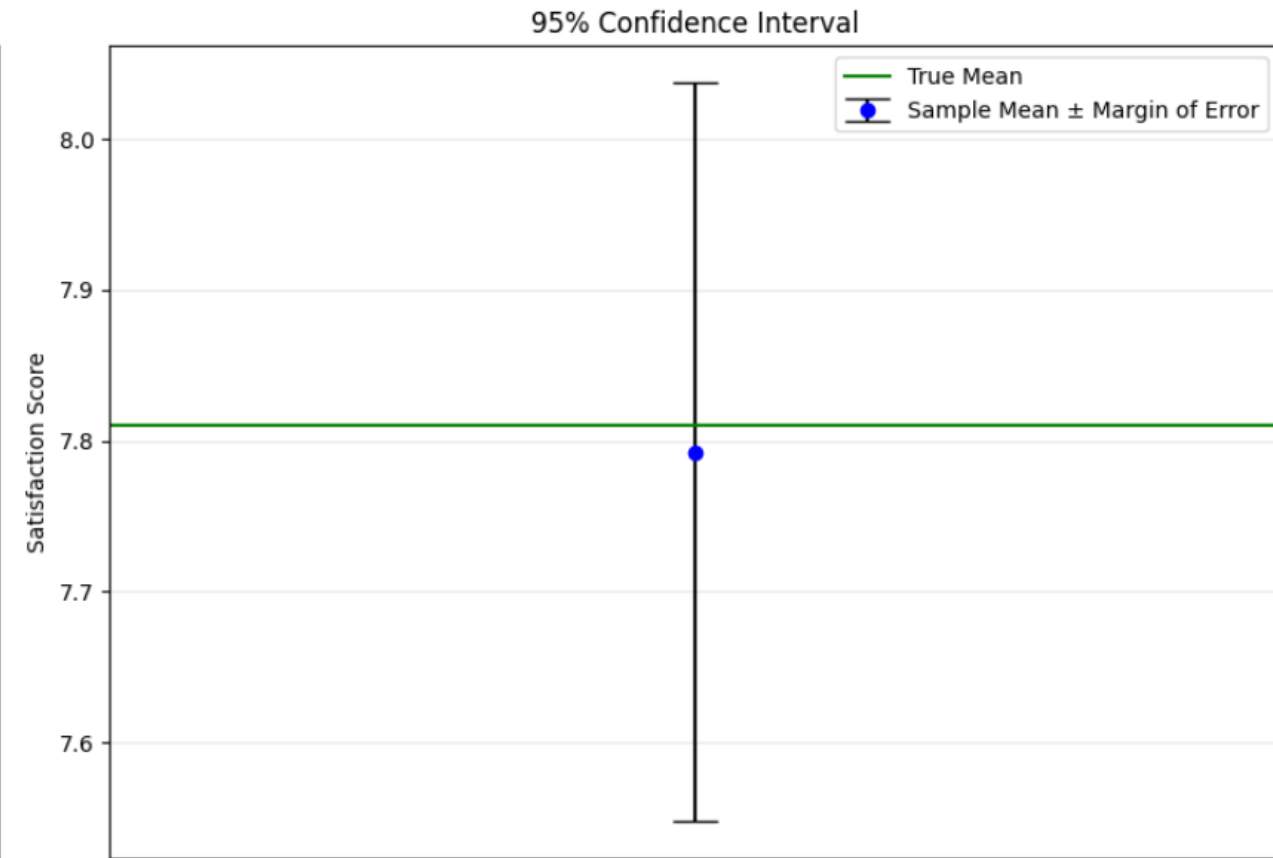
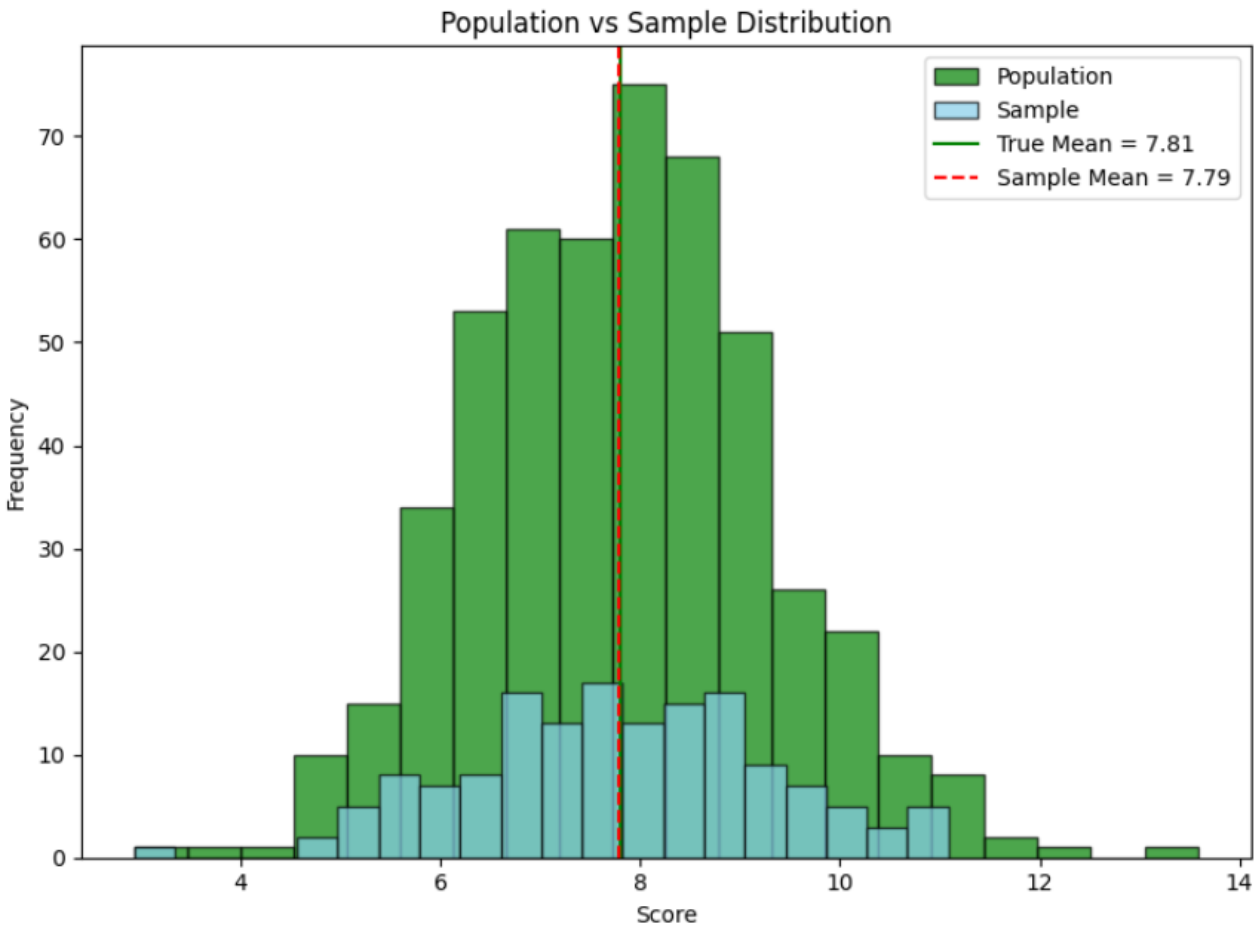
```
plt.show()
```



Confidence level: 95%
t-critical value (df=149): 1.9760
Margin of error: ± 0.2446

95% Confidence Interval: [7.55, 8.04]

Interpretation: We are 95% confident that the true average satisfaction score for ALL hotel guests is between 7.55 and 8.04.



4. One-Tailed vs Two-Tailed Tests

Two-Tailed Test: The "Any Difference" Test

Goal: You want to know if a result is just **different** (could be higher OR lower).

- Null hypothesis: $H_0: \mu = \mu_0$
- Alternative hypothesis: $H_1: \mu \neq \mu_0$
- You **split** the significance level α **equally between both tails**.
- For 95% confidence: $\alpha = 0.05 \rightarrow$ each tail = 0.025 $\rightarrow$ cumulative cutoff = **0.975**
- Decision rule: **reject if $|t| > t_{\text{critical}}$**

Test Type	Confidence Level	Cumulative Cutoff for <code>ppf()</code>
Two-tailed	95%	0.975
One-tailed	95%	0.95

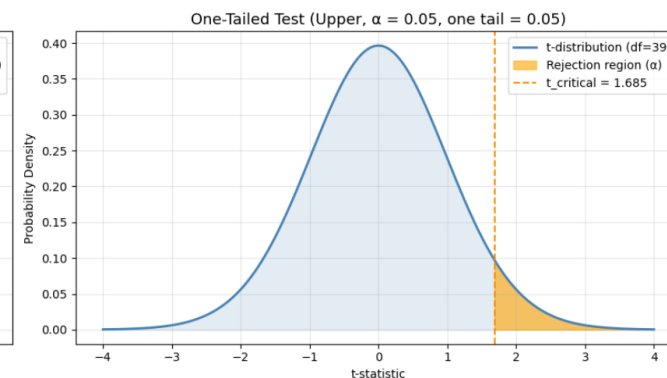
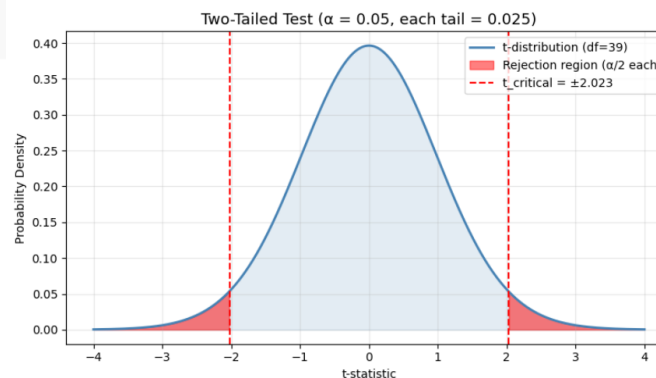
```
t_critical = t.ppf(0.975, df)
```

One-Tailed Test: The "Directional" Test

Goal: You only care if a result is **better** (or specifically worse).

- Null hypothesis: $H_0: \mu \leq \mu_0$ (or $H_0: \mu \geq \mu_0$)
- Alternative hypothesis: $H_1: \mu > \mu_0$ (or $H_1: \mu < \mu_0$)
- **All** α is placed in **one tail**.
- For 95% confidence: $\alpha = 0.05 \rightarrow$ one tail = 0.05 $\rightarrow$ cumulative cutoff = **0.95**
- Decision rule: **reject if $t > t_{\text{critical}}$** (for upper-tailed) or **$t < -t_{\text{critical}}$** (for lower-tailed)

```
t_critical = t.ppf(0.95, df)
```



```
x = np.linspace(-4, 4, 1000)
```

```
y = stats.t.pdf(x, df)
```

```
ax.plot(x, y, 'steelblue', linewidth=2, label='t-distribution (df=39)')
```

```
ax.fill_between(x, y, alpha=0.15, color='steelblue')
```

```
t_crit_2 = stats.t.ppf(0.975, df) # cutoff = 0.975 for two-tailed 95%
```

```
# Shade both rejection regions
```

```
ax.fill_between(x[x <= -t_crit_2], y[x <= -t_crit_2], color='red', alpha=0.15)
```

```
ax.fill_between(x[x >= t_crit_2], y[x >= t_crit_2], color='red', alpha=0.15)
```

```
ax.axvline(-t_crit_2, color='red', linestyle='--', linewidth=1.5)
```

```
ax.axvline(t_crit_2, color='red', linestyle='--', linewidth=1.5, label=
```

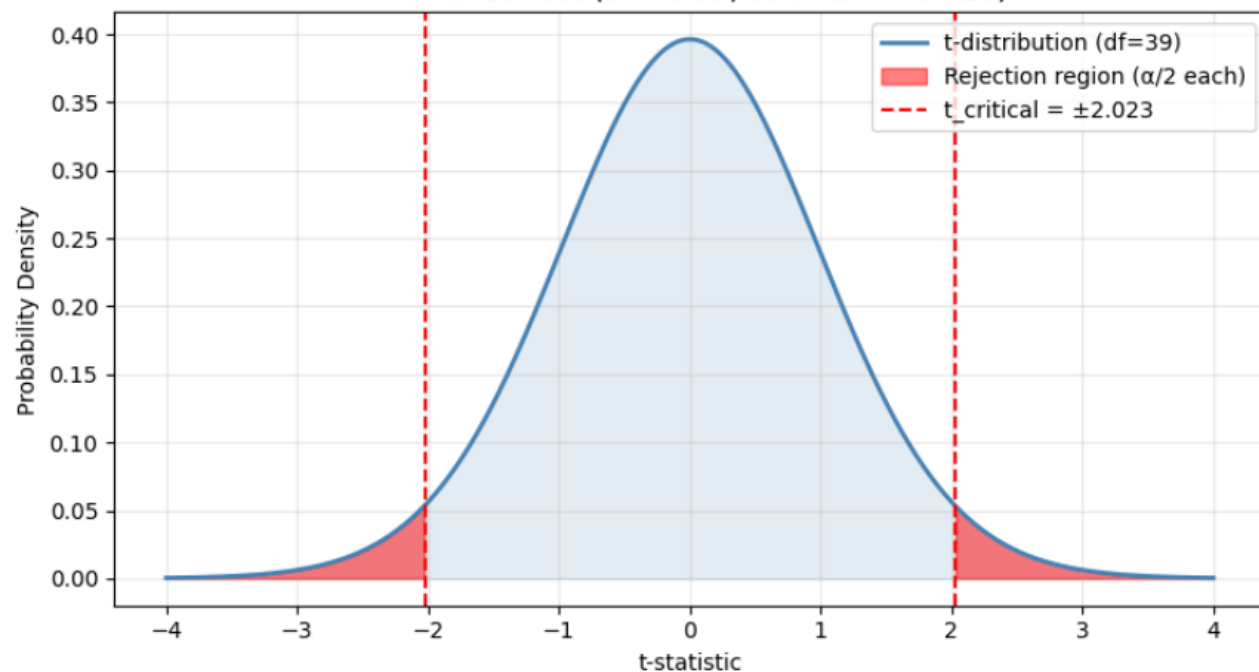
Two-tailed t_{critical} (df=39, 95% CI): ± 2.0227

One-tailed t_{critical} (df=39, 95%): 1.6849

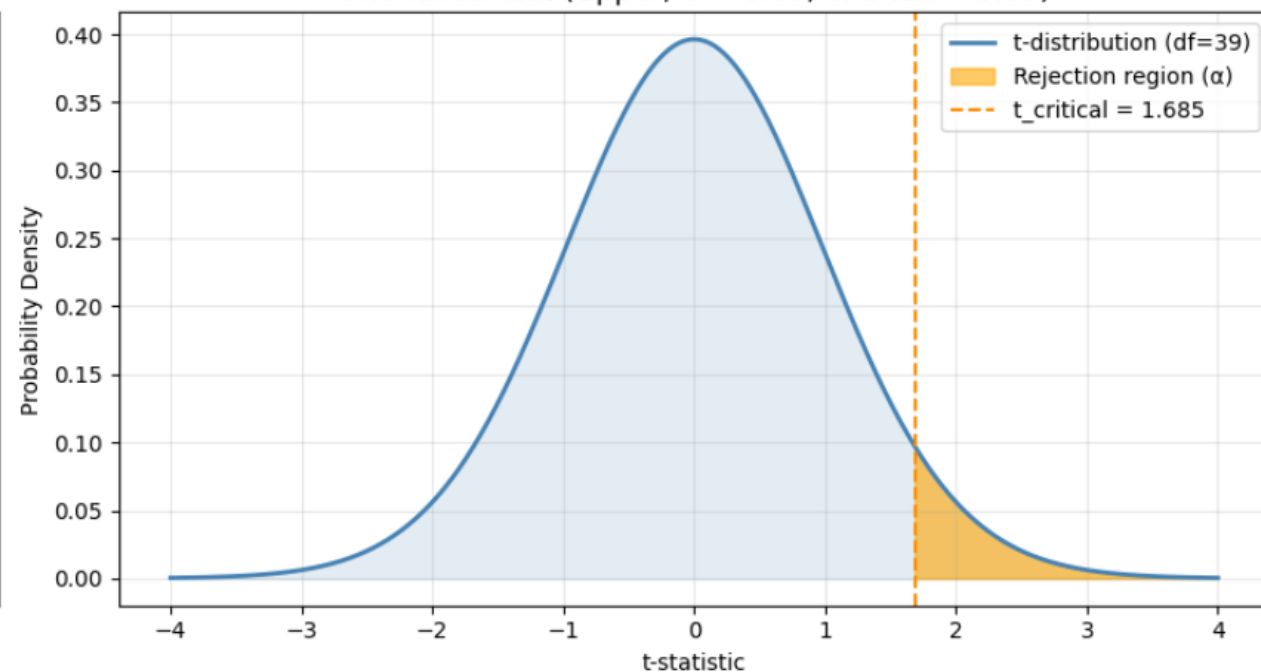
- `stats.t.pdf(x, df)` – **Probability Density Function** of the t-distribution with `df` degrees of freedom. Gives the height of the bell curve at each `x` value.
- `stats.t.ppf(q, df)` – **Percent Point Function** (inverse CDF). Returns the `t`-value such that fraction `q` of the distribution is below it.
 - Two-tailed: `ppf(0.975, df)` because each tail has 2.5%.
 - One-tailed: `ppf(0.95, df)` because one tail has 5%.

```
t_crit_1 = stats.t.ppf(0.95, df) # cutoff = 0.95 for one-tailed 95%
```

Two-Tailed Test ($\alpha = 0.05$, each tail = 0.025)



One-Tailed Test (Upper, $\alpha = 0.05$, one tail = 0.05)



5. Type I and Type II Errors

These are the two kinds of mistakes in statistical hypothesis testing:

	H_0 is actually TRUE	H_0 is actually FALSE
Reject H_0	✗ Type I Error (False Positive)	✓ Correct (True Positive)
Fail to Reject H_0	✓ Correct (True Negative)	✗ Type II Error (False Negative)

Reject H_0 = +ve → Good H_1
-ve

Type I Error (False Positive)

Definition: Wrongly rejecting a true null hypothesis — saying there's an effect when there **isn't**.

- **Example:** A drug has no effect, but your test concludes it does. You approve a useless drug.

Type II Error (False Negative)

Definition: Failing to reject a false null hypothesis — missing an effect that's **actually there**.

- **Example:** A new teaching method genuinely works, but your test fails to detect it. A valuable innovation is abandoned.

6. Choosing the Right Hypothesis Test

Scenario	Question	Test
One sample	Does a sample mean differ from a known value?	One-Sample t-test
Two independent groups (means)	Are Group A & B means different?	Independent Samples t-test
Two paired groups (means)	Are "Before" & "After" scores different?	Paired t-test
3+ independent groups (means)	Are Treatment 1, 2, 3 means different?	One-Way ANOVA
Two categorical variables	Is there a link between Gender & Pass/Fail?	Chi-Squared Test

7. T-Tests

T-tests compare means.

7.1 One-Sample T-Test

The core idea: the **t-statistic** measures how many standard errors the sample mean is away from the hypothesized population mean.

$$t = (\bar{x} - \mu_0) / (s / \sqrt{n})$$

- $\bar{x}$ = sample mean
- μ_0 = hypothesized population mean (from H_0)
- s = sample standard deviation

Reading the t-Table

The t-table gives you the **critical value** — the threshold beyond which you reject H_0 .

If $|t_{\text{statistic}}| > t_{\text{critical}}$, reject H_0 . Equivalently, if **p-value** $< \alpha$, reject H_0 .

- Confidence interval for the sample mean: $\bar{X} \pm (t_{\text{critical}} \times SE)$

Using One-Sample T-Test in ML Research Papers

In machine learning research, the one-sample t-test is used to **prove that a model's performance is significantly different from a known baseline or threshold**, rather than attributing the result to random variation due to the data split.

Classification research:

Suppose your classification model achieves a mean accuracy of 87.4% across 10 cross-validation folds. You want to prove it significantly outperforms a known industry baseline of 85%.

- **H₀:** $\mu_{\text{accuracy}} = 0.85$ (the model's true accuracy equals the baseline)
- **H₁:** $\mu_{\text{accuracy}} \neq 0.85$ (the model significantly differs from the baseline)
- You apply a one-sample t-test on the 10 fold accuracies against the value 0.85.
- If $p < 0.05 \rightarrow$ your model is statistically significantly different from the baseline. You can claim this result in a paper.

Regression research:

Your regression model yields a mean RMSE of 4.2 across 10 folds. The published state-of-the-art reports an RMSE of 5.0. You want to verify the improvement is not by chance.

- **H₀:** $\mu_{\text{RMSE}} = 5.0$ (your model is no better than the benchmark)
- **H₁:** $\mu_{\text{RMSE}} < 5.0$ (one-tailed: your model has significantly lower error)
- A one-sample t-test on fold RMSE values against 5.0 gives statistical backin

Why this matters in papers: Reviewers often ask "is this improvement statistically significant or just lucky data splits?" The one-sample t-test provides rigorous evidence.

Real-World Case: Quality Control

A beverage company claims bottles contain 500 ml. Regulatory inspectors test 30 bottles to verify compliance.

- **Null hypothesis H_0 :** $\mu = 500$ ml $\rightarrow$ Bottles meet specifications.
- **Alternative hypothesis H_1 :** $\mu \neq 500$ ml $\rightarrow$ Bottles do NOT meet specifications.
- **$\alpha = 0.05$** $\rightarrow$ 95% confidence level
- **$df = n - 1 = 29$**

```
target_volume = 500 # claimed/target value ( $\mu_0$ )
# Simulating 30 bottles filled at mean=498 (slightly underfilled), std=5
measured_volumes = np.random.normal(498, 5, 30)
```

```
# ttest_1samp(a, popmean): tests if mean of array a equals popmean
t_statistic, p_value = ttest_1samp(measured_volumes, target_volume)
```

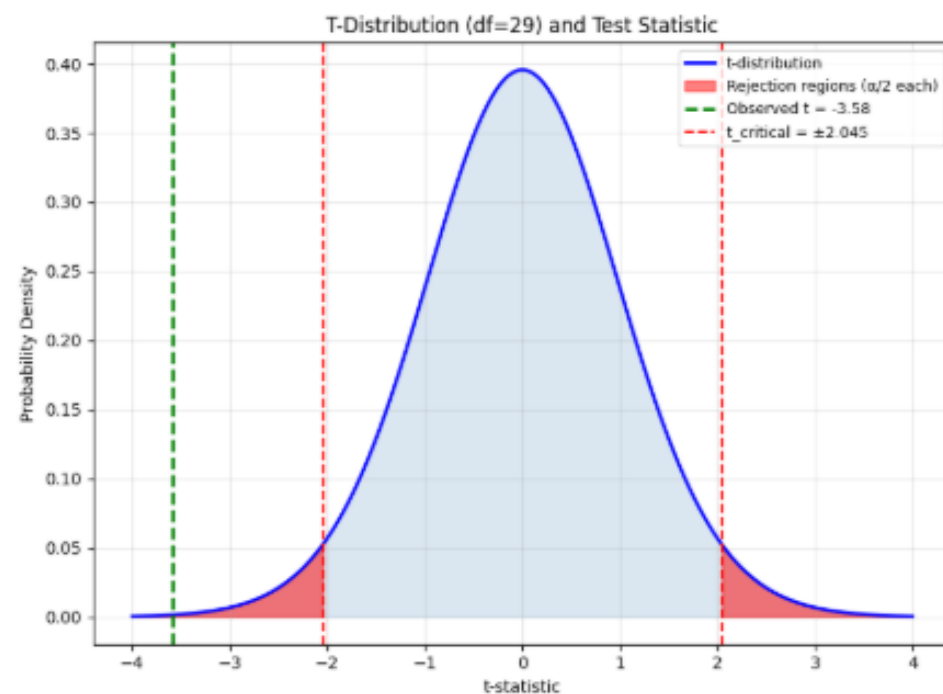
```
sample_mean = np.mean(measured_volumes)
sample_std = np.std(measured_volumes, ddof=1)
n = len(measured_volumes)
se = sample_std / np.sqrt(n)
df = n - 1
```

```
# 0.975 → two-tailed 95% CI: middle 95% leaves 2.5% in each tail
```

```
t_crit = stats.t.ppf(0.975, df)
ci_lower = sample_mean - t_crit * se
ci_upper = sample_mean + t_crit * se
```

Interpreting the t-distribution plot:

- The bell curve is the t-distribution under H_0 .
- Red shaded regions = rejection zones (extreme values beyond $\pm t_{\text{critical}}$).
- Green dashed line = observed t-statistic.
- If the green line falls in the red zone → reject H_0 .
- Note: The 95% CI on the left plot does NOT contain 500 ml → confirms rejection of H_0 .



```
axes[0].hist(  
    measured_volumes,  
    bins=15,
```

✗ REJECT H_0

Test Results:

t-statistic: -3.5793

t-critical (df=29, two-tailed 95%): 2.0452

p-value: 0.0012

95% CI: [495.38, 498.74] ml

```
axes[0].axvline(  
    sample_mean,  
    color='blue',
```

```
axes[0].axvline(  
    target_volume,  
    color='red',
```

```
axes[0].axvline(ci_lower, color='green',  
axes[0].axvline(ci_upper, color='green',
```

```
x = np.linspace(-4, 4, 1000)  
y = stats.t.pdf(x, df)
```

```
axes[1].plot(x, y,
```

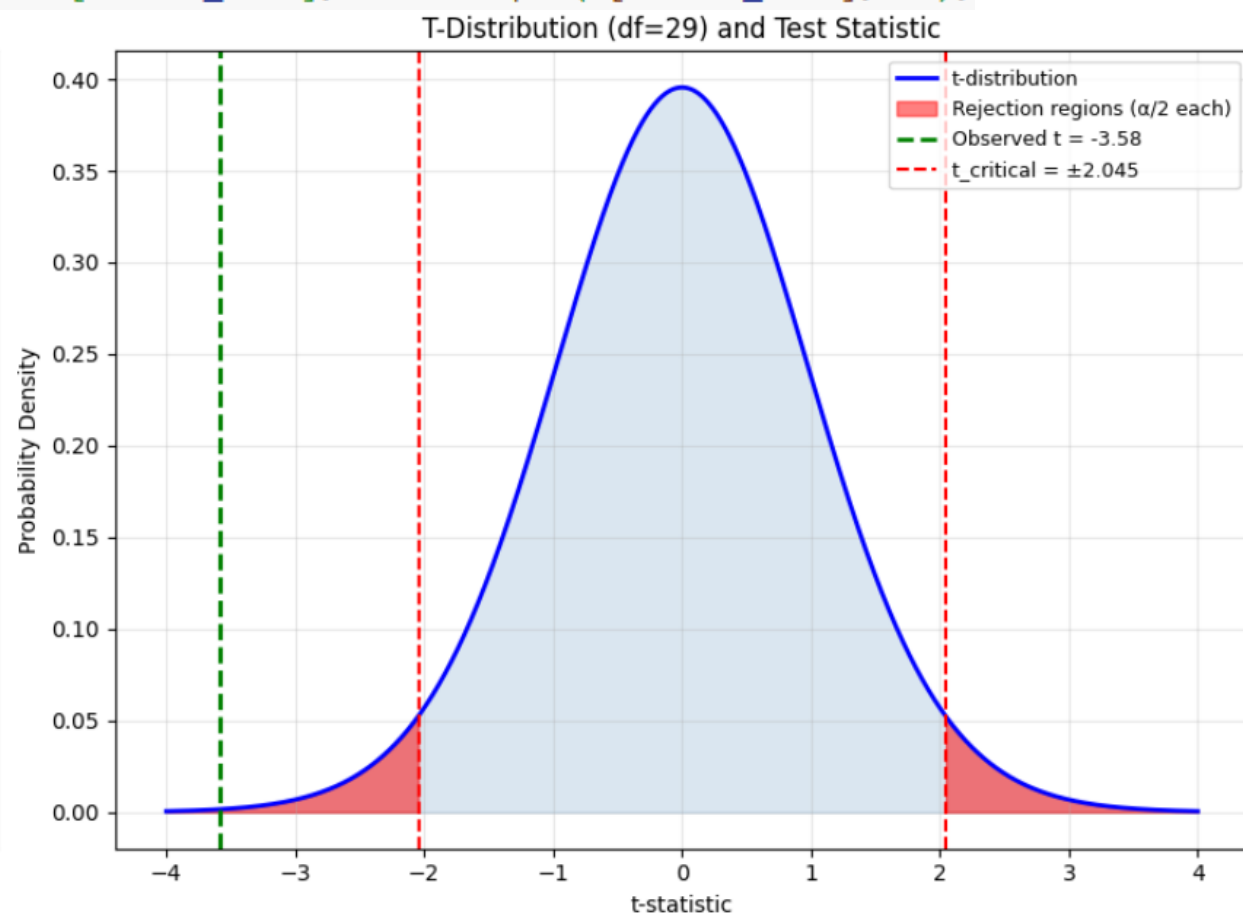
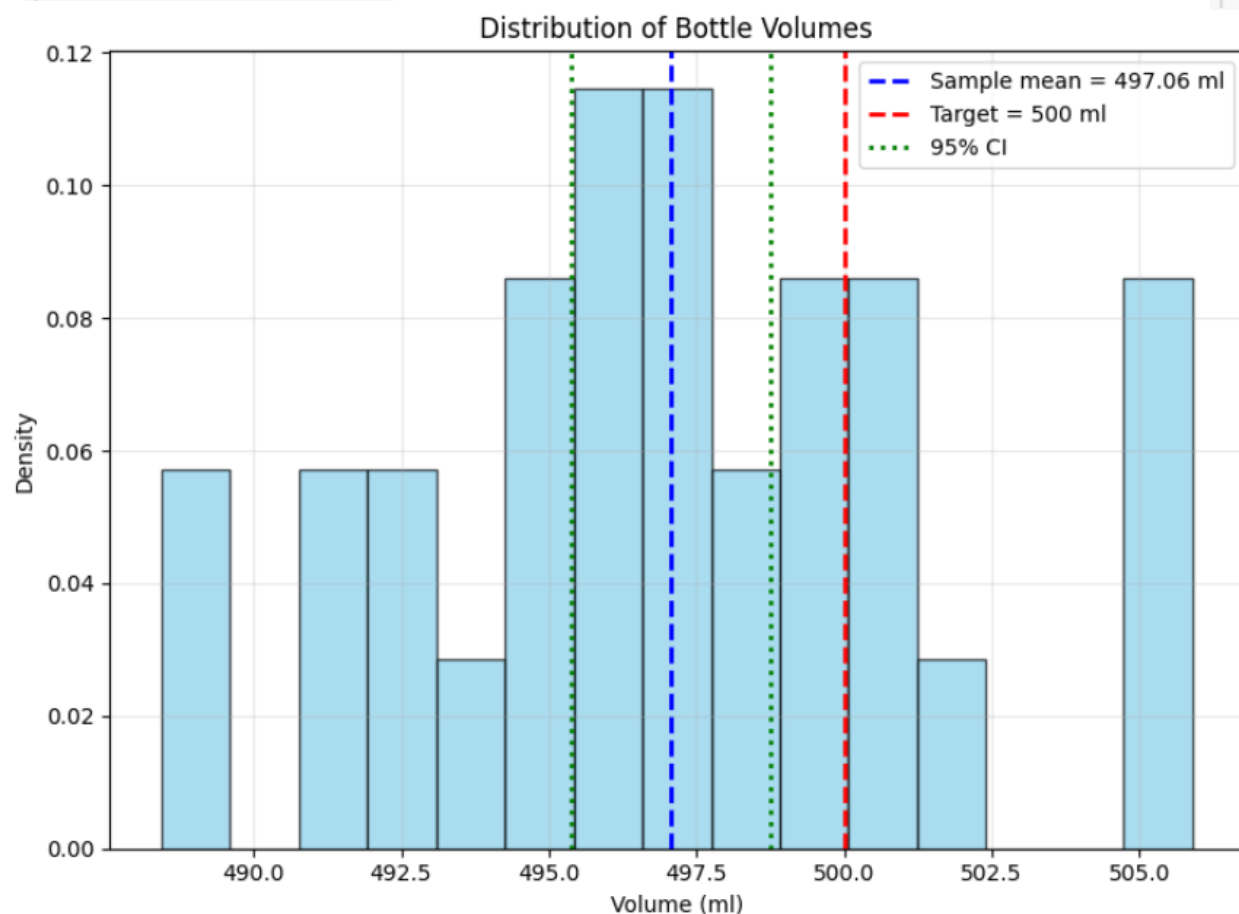
```
axes[1].fill_between(x, y,
```

```
axes[1].fill_between(  
    x[x <= -t_crit], stats.t.pdf(x[x <= -t_crit], df),
```

```
axes[1].fill_between(  
    x[x >= t_crit], stats.t.pdf(x[x >= t_crit], df),
```

```
axes[1].axvline(  
    t_statistic,  
    color='green',
```

```
axes[1].axvline(-t_crit, color='red',  
axes[1].axvline(t_crit, color='red',
```



Example: Fail to Reject H_0 (Bottles PASS)

Now let's simulate bottles that are actually filled at 500 ml (no defect). We should fail to reject H_0 .

```
target_volume_pass = 500
pass_volumes = np.random.normal(500, 5, 30) # mean = target → no defect
```

One-Sample T-Test: Bottles PASS scenario

=====

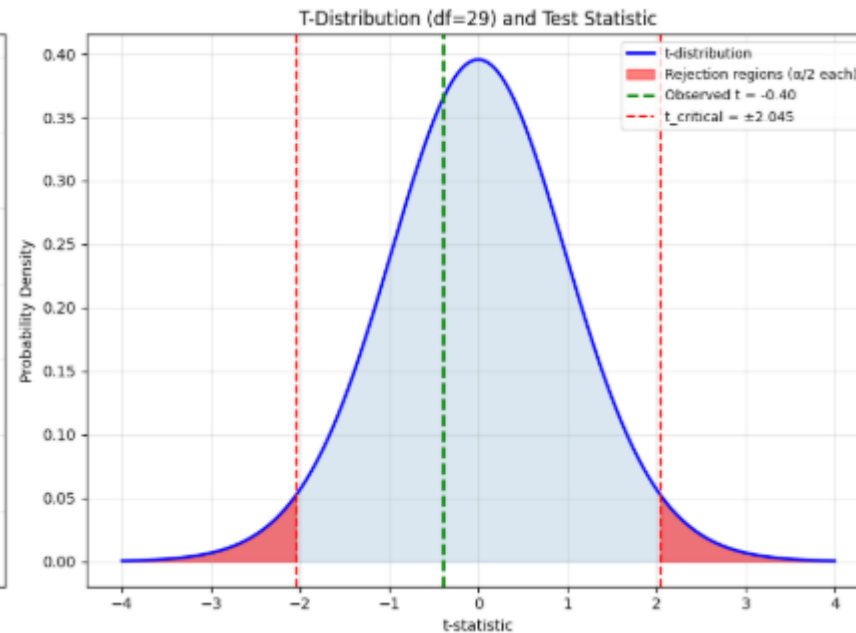
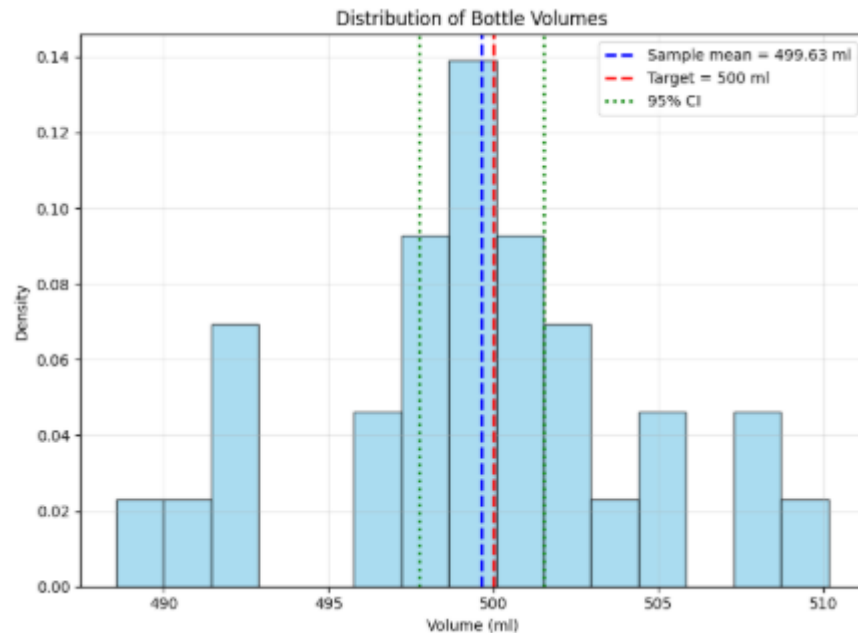
Sample mean: 499.63 ml (target: 500 ml)

t-statistic: -0.3959

t-critical: ± 2.0452

p-value: 0.6950

✓ FAIL TO REJECT H_0 – bottles meet specifications (PASS)



ML Example — Reject H_0 : Model Accuracy Significantly Exceeds Baseline

A classification model is trained and evaluated on 10 cross-validation folds. The published baseline accuracy is **0.80 (80%)**. We want to confirm that the model's mean accuracy is significantly above this threshold — not just a lucky split.

- H_0 : $\mu_{\text{accuracy}} = 0.80$ (not better than baseline)
- H_1 : $\mu_{\text{accuracy}} \neq 0.80$ (significantly different from baseline)
- $\alpha = 0.05$

One-Sample T-Test: ML Model Accuracy vs Published Baseline

=====

H_0 : $\mu_{\text{accuracy}} = 0.8$ (no improvement over baseline)

H_1 : $\mu_{\text{accuracy}} \neq 0.8$ (significantly different)

10-fold CV accuracies: [0.9 0.8889 0.8482 0.8758 0.8872 0.863 0.8808 0.878 0.8761 0.8729]

Sample mean accuracy: 0.8771

Std dev: 0.0143 | SE: 0.0045

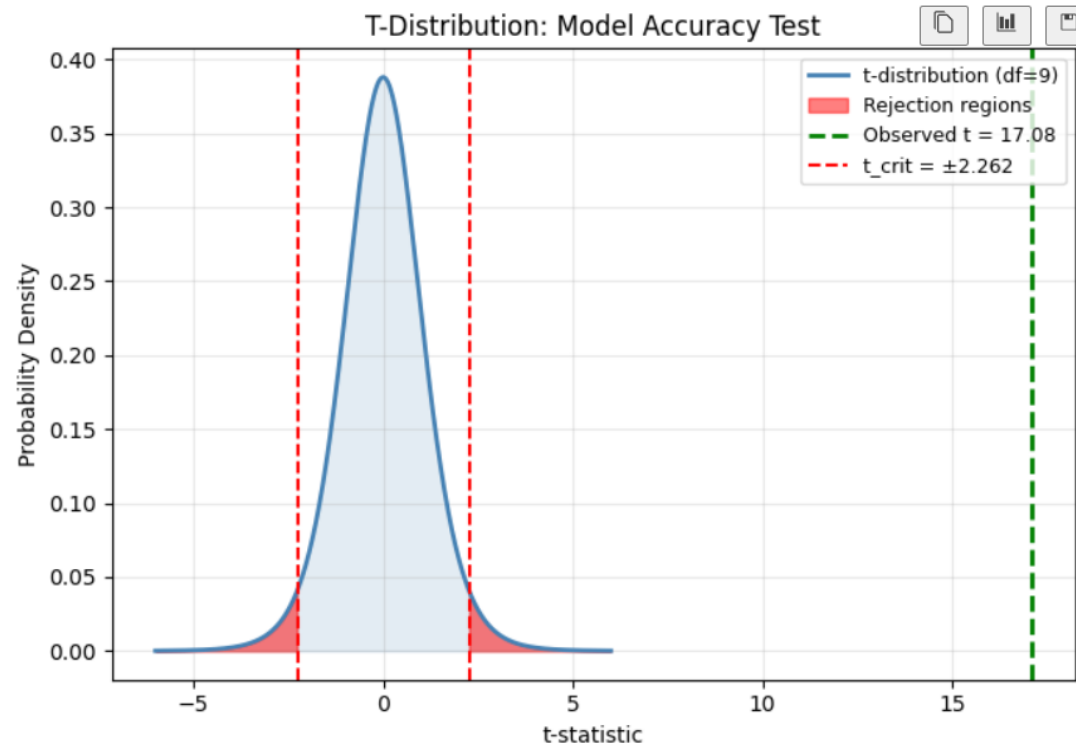
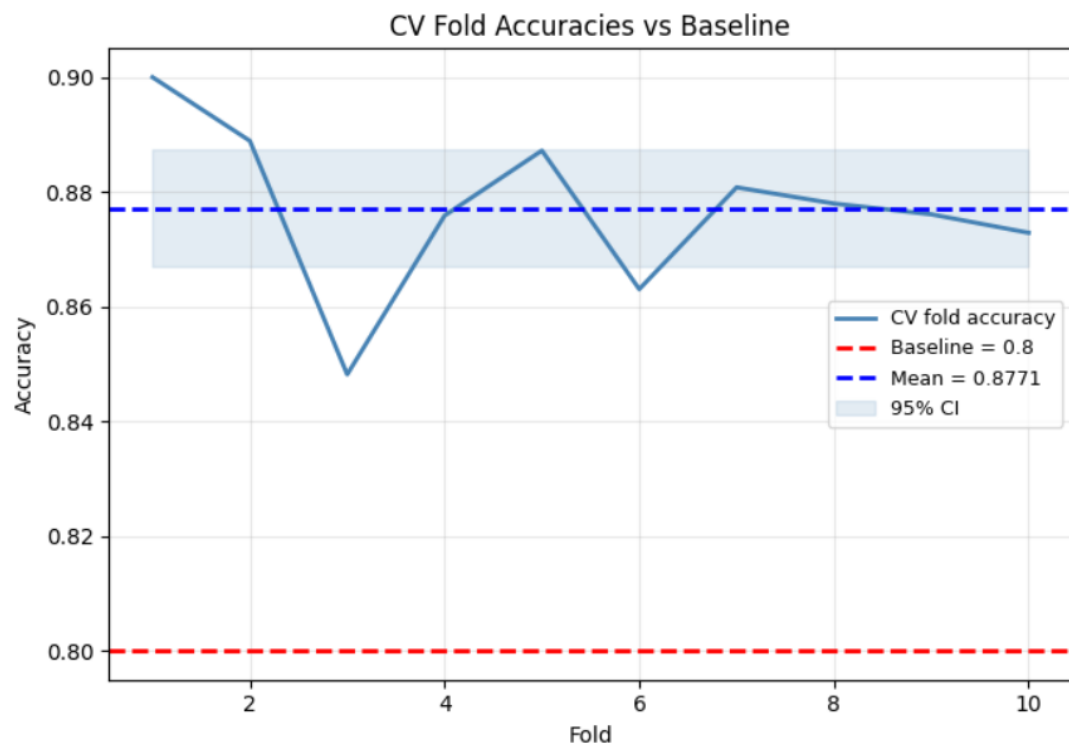
t-statistic: 17.0831
t-critical (df=9): ± 2.2622
p-value: 0.000000
95% CI: [0.8669, 0.8873]

✓ REJECT H_0 ($p < 0.05$)

The model's accuracy (87.71%) is statistically significantly different from the baseline (80.00%).

The CI [86.69%, 88.73%] lies entirely above baseline.

→ Paper claim: Our model significantly outperforms the baseline ($p=0.0000$).



ML paper reporting format:

"The proposed model achieved a mean 10-fold CV accuracy of 87.6% (95% CI: [86.5%, 88.7%]), significantly exceeding the published baseline of 80% (one-sample t-test: $t(9) = XX$, $p < 0.001$)."

ML Example — Fail to Reject H_0 : Model Performance Not Significantly Above Baseline

One-Sample T-Test: Regression Model RMSE vs Benchmark

=====

H_0 : $\mu_{\text{RMSE}} = 4.5$ (no improvement over benchmark)

H_1 : $\mu_{\text{RMSE}} \neq 4.5$

10-fold RMSE values: [4.3924 3.981 4.7445 4.3482 4.2727 4.1193 4.6956 4.0889 4.6079 4.2515]

Mean RMSE: 4.3502 (benchmark: 4.5)

t-statistic: -1.8095

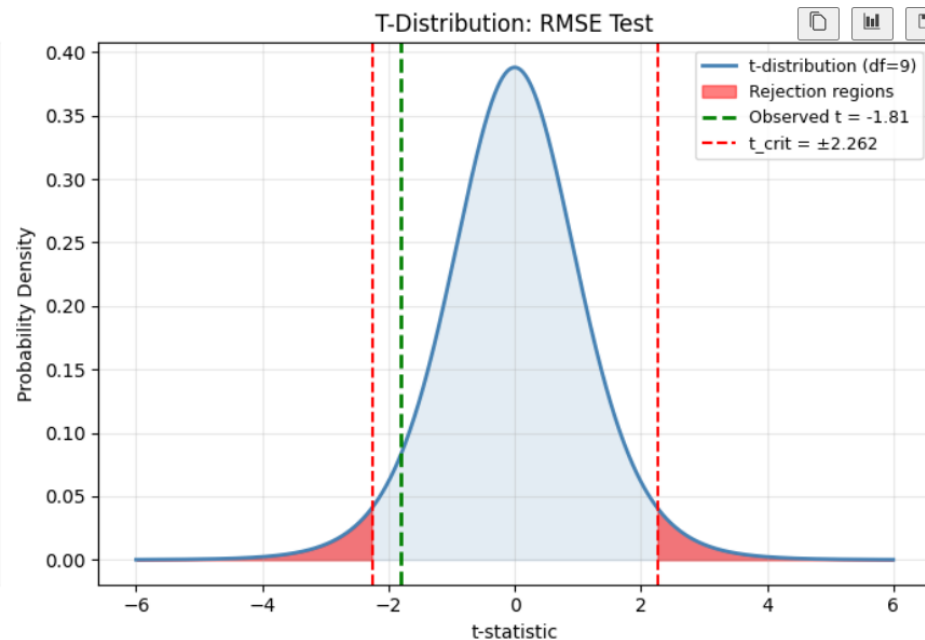
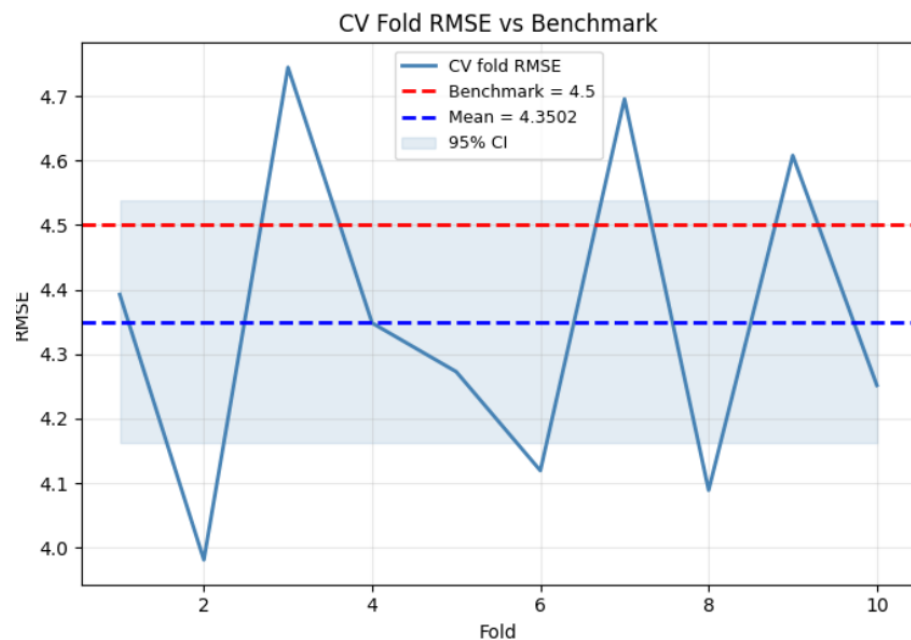
p-value: 0.1038

✗ FAIL TO REJECT H_0 ($p = 0.1038 \geq 0.05$)

Although mean RMSE (4.3502) < benchmark (4.5),
the difference is NOT statistically significant.

→ Paper claim: We cannot claim our model outperforms the benchmark ($p=0.1038$).

→ Action: Collect more data or improve the model before claiming superiority.



Code Walkthrough

- Even though the mean RMSE is slightly lower than the benchmark, the **high variability across folds** (std ≈ 0.30) means the difference could easily be due to chance.
- The t-statistic is small and the p-value is large $\rightarrow$ we cannot confidently say the model beats the benchmark.
- This is a critical finding in ML research: **do not claim improvement without statistical backing**. A lower number on the test set alone is not enough — it must be significantly lower.

ML paper reporting format:

"The proposed model's mean RMSE of 4.42 across 10 folds did not significantly differ from the benchmark RMSE of 4.50 (one-sample t-test: $t(9) = XX$, $p = 0.XX$, n.s.), suggesting the improvement may be attributable to variance in data splits rather than model superiority."

7.2 Two-Sample T-Test (Independent)

When to use: Compare means of **two independent groups** — groups with no inherent pairing between their members.

Real-World Case: Marketing Campaign Comparison

A company runs two marketing campaigns (A and B) in different regions. The goal: compare average sales between the two campaigns to see if one performs significantly better.

- $H_0: \mu_A = \mu_B \rightarrow$ both campaigns have the same average sales.
- $H_1: \mu_A \neq \mu_B \rightarrow$ campaigns differ in average sales.
- $\alpha = 0.05, df = n_1 + n_2 - 2$

$$t = \frac{\bar{X}_1 - \bar{X}_2}{SE}$$

$$SE = \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}$$

- $(\bar{X}_1, \bar{X}_2)$ = sample means of groups 1 and 2.
- (s_1^2, s_2^2) = sample variances of groups 1 and 2.
- (n_1, n_2) = sample sizes of groups 1 and 2.
- Reject if $|t| > t_{critical}$.
- Confidence Interval for Difference in Means = $(\bar{X}_1 - \bar{X}_2) \pm (t_{critical} \times SE)$
- Reject if CI doesn't include 0.

$$df = n_1 + n_2 - 1 - 1$$

```
# Campaign A: mean ≈ $5000, std dev ≈ $1200, 50 samples
campaign_a_sales = np.random.normal(5000, 1200, 50)
# Campaign B: mean ≈ $5500, std dev ≈ $1300, 50 samples
campaign_b_sales = np.random.normal(5500, 1300, 50)
```

```
# — Step 2: Two-sample t-test —————
# ttest_ind(a, b): independent-samples t-test (two-tailed by default)
t_stat, p_value = ttest_ind(campaign_a_sales, campaign_b_sales)
```

```
t_critical = st.t.ppf(0.975, df_2)
```

```
# — Step 3: Compute statistics —————
mean_a = np.mean(campaign_a_sales)
mean_b = np.mean(campaign_b_sales)
std_a = np.std(campaign_a_sales, ddof=1)
std_b = np.std(campaign_b_sales, ddof=1)
n_a = len(campaign_a_sales)
n_b = len(campaign_b_sales)
df_2 = n_a + n_b - 2
```

Two-Sample T-Test: Marketing Campaign Comparison
=====

$H_0: \mu_A = \mu_B$ (same average sales)
 $H_1: \mu_A \neq \mu_B$ (different average sales)

Campaign A (n=50): Mean = \$4,729.43, Std = \$1,120.40
Campaign B (n=50): Mean = \$5,523.12, Std = \$1,136.62

Test Results:

- t-statistic: -3.5164
- t-critical (df=98, two-tailed 95%): ±1.9845
- p-value: 0.0007

✓ REJECT H_0 ($p < 0.05$): Campaign B significantly outperforms A
Business decision: Expand Campaign B strategy

```
bp = axes[0].boxplot(
    [campaign_a_sales, campaign_b_sales],
    tick_labels=['Campaign A', 'Campaign B'],
    patch_artist=True,
)
axes[0].plot(1, mean_a,
axes[0].plot(2, mean_b,
```

```
axes[1].hist(campaign_a_sales, bins=20,
axes[1].hist(campaign_b_sales, bins=20,

axes[1].axvline(mean_a, color='blue',
axes[1].axvline(mean_b, color='green',
```

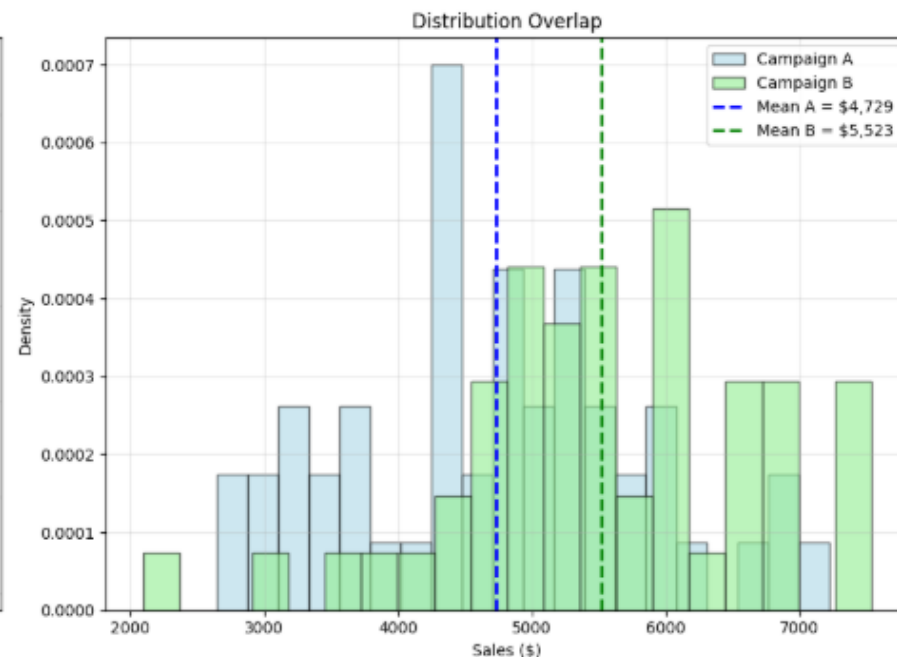
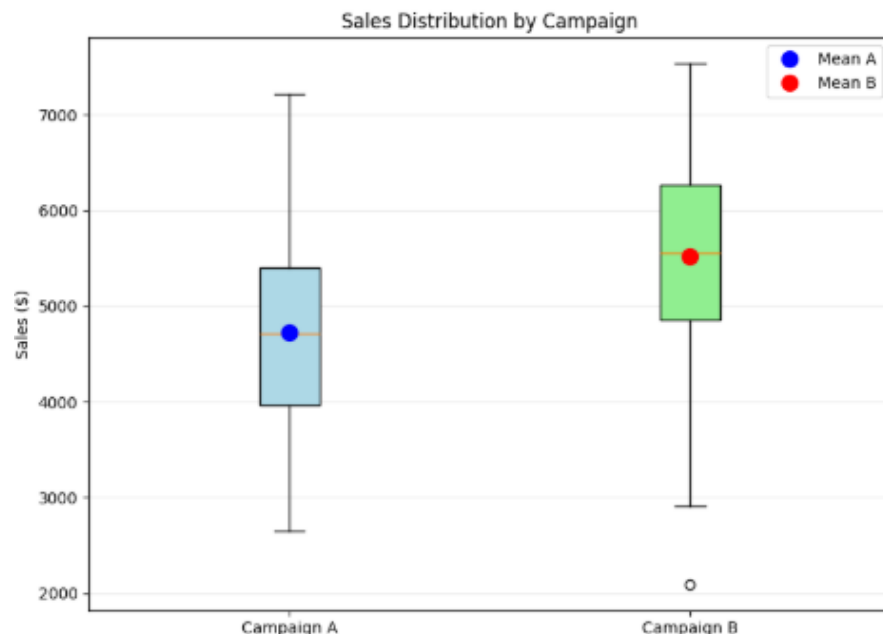
Test Results:

t-statistic: -3.5164

t-critical (df=98, two-tailed 95%): ± 1.9845

p-value: 0.0007

✓ REJECT H_0 ($p < 0.05$): Campaign B significantly outperforms A
Business decision: Expand Campaign B strategy



Box plot interpretation: The boxes show the interquartile range (25th–75th percentiles). The red dots mark the group means. If the boxes overlap heavily, differences may not be significant; if clearly separated, one campaign likely outperforms.

Example: Fail to Reject H_0 (No significant difference)

Two-Sample T-Test: No Significant Difference scenario

=====

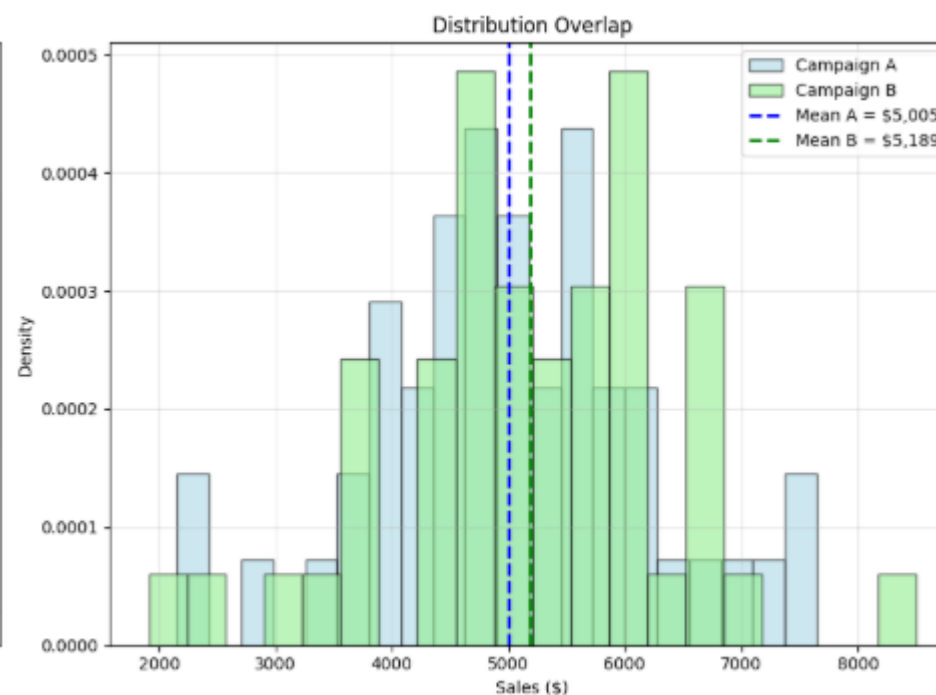
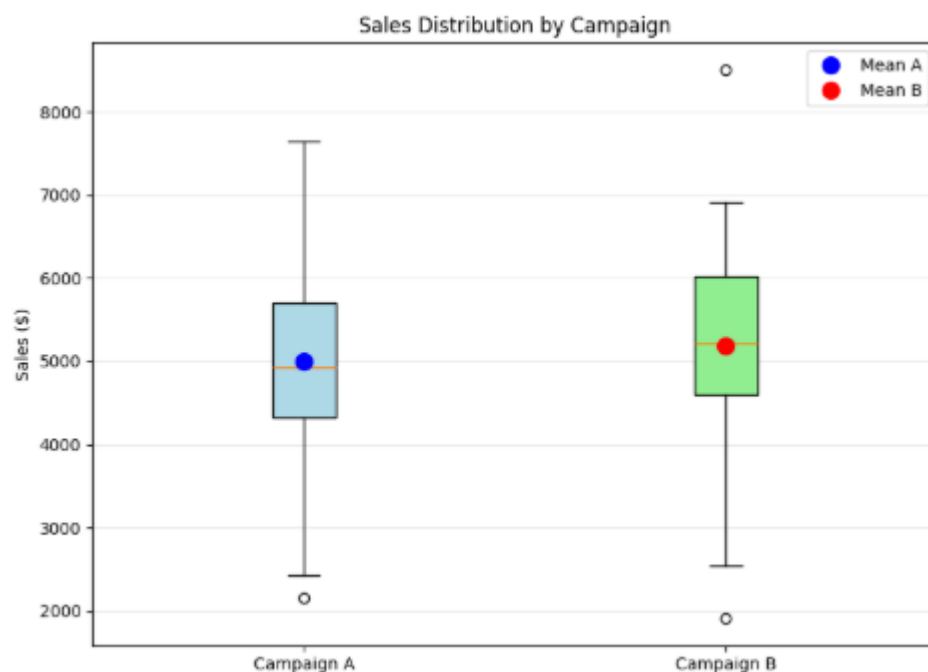
Mean A: \$5,004.98 | Mean B: \$5,188.59

t-statistic: -0.7658

p-value: 0.4456

✗ FAIL TO REJECT H_0 – no significant difference between campaigns

Business decision: Either campaign is acceptable



ML Example — Reject H_0 : New Model Significantly Outperforms

Baseline Model

Two classification models are trained and evaluated independently on 15 random train/test splits each. We want to test if Model B (a new architecture) significantly outperforms Model A (the current production model).

t-statistic: -5.0838

t-critical (df=28): ± 2.0484

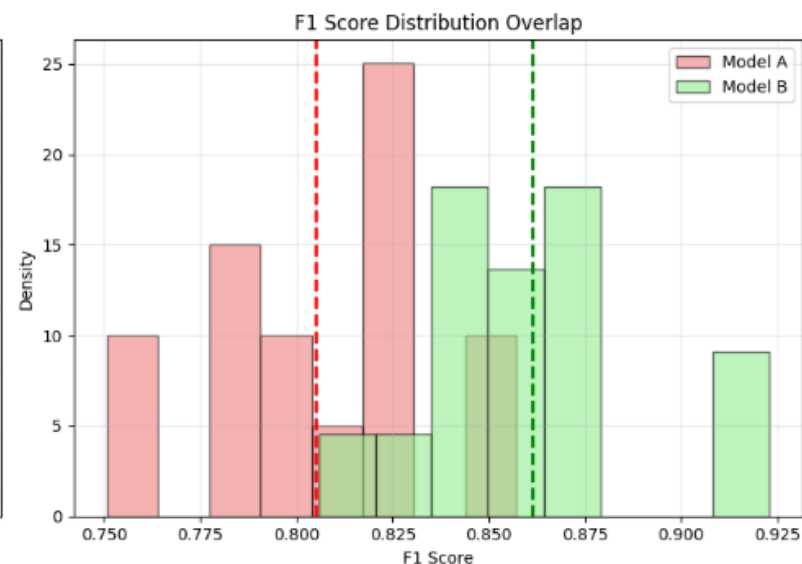
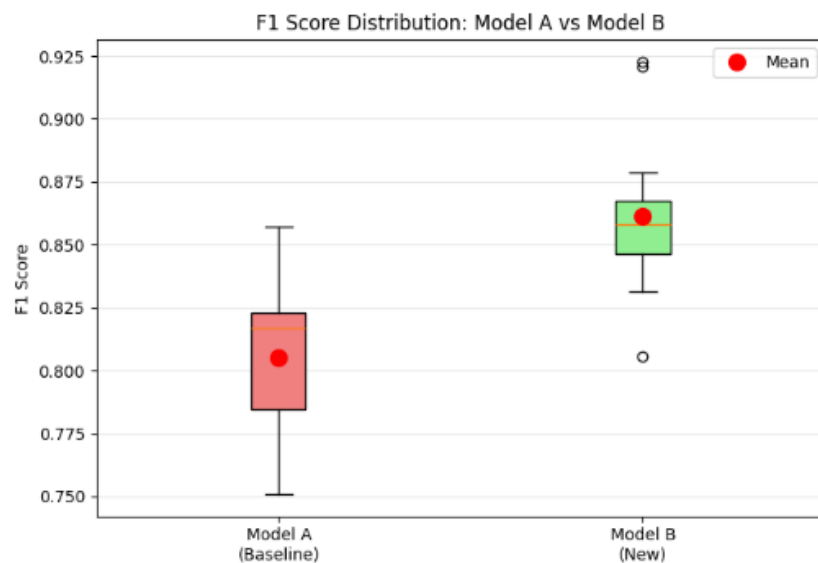
p-value: 0.000022

✓ REJECT H_0 ($p < 0.05$)

Model B (86.12%) significantly outperforms

Model A (80.51%) in F1 score.

→ Paper claim: Model B shows statistically significant improvement ($p=0.0000$).



7.3 Paired T-Test

When to use: Compare means of the **same group** at two different times (before/after) or under two different conditions.

Why paired? Each "after" measurement is directly related to its "before" measurement for the **same individual**. It is essentially a one-sample t-test on the **differences** ($d = \text{after} - \text{before}$), testing whether the mean difference equals 0.

$$t = \text{mean}(d) / (s_d / \sqrt{n})$$

Where s_d = standard deviation of the differences.

Real-World Case: Training Program Effectiveness

A company measures productivity for the **same 40 employees** before and after a training program.

- $H_0: \mu_{\text{diff}} = 0 \rightarrow$ Training has no effect.
- $H_1: \mu_{\text{diff}} > 0 \rightarrow$ Training improves productivity.
- $\alpha = 0.05, df = n - 1 = 39$

Paired T-Test: Training Program Effectiveness

=====

$H_0: \mu_{\text{diff}} = 0$ (training has no effect)

$H_1: \mu_{\text{diff}} > 0$ (training improves productivity)

```
t_stat, p_value = ttest_rel(productivity_after, productivity_before)
```

```
improvement = productivity_after - productivity_before
```

```
mean_improvement = np.mean(improvement)
```

```
std_improvement = np.std(improvement, ddof=1)
```

```
se_improvement = std_improvement / np.sqrt(n_employees)
```

```
df_p = n_employees - 1
```

```
t_crit_p = stats.t.ppf(0.975, df_p)
```

```
ci_lower_p = mean_improvement - t_crit_p * se_improvement
```

```
ci_upper_p = mean_improvement + t_crit_p * se_improvement
```

✓ REJECT H_0 ($p < 0.05$): Training significantly improves productivity

Recommendation: Implement training company-wide

Test Results:

t-statistic: 10.2984

t-critical (df=39): ± 2.0227

p-value: 0.0000

95% CI for improvement: [6.31, 9.40]

`ttest_rel(a, b)` – paired (related-samples) t-test.

- `a` and `b` must have the **same length** (one measurement per individual in each condition).
- Internally computes `d = a - b` and tests if `mean(d) == 0`.

df = n - 1 (not $n_1 + n_2 - 2$ like the two-sample test) because we only have one group of differences.


```
# — Plot 1: Paired before vs after —
```

```
for b, a in zip(productivity_before, productivity_after):  
    axes[0, 0].plot([0, 1], [b, a], 'o-', alpha=0.3, color='gray')
```

```
axes[0, 0].plot([0, 1], [np.mean(productivity_before), np.mean(productivity_after)],
```

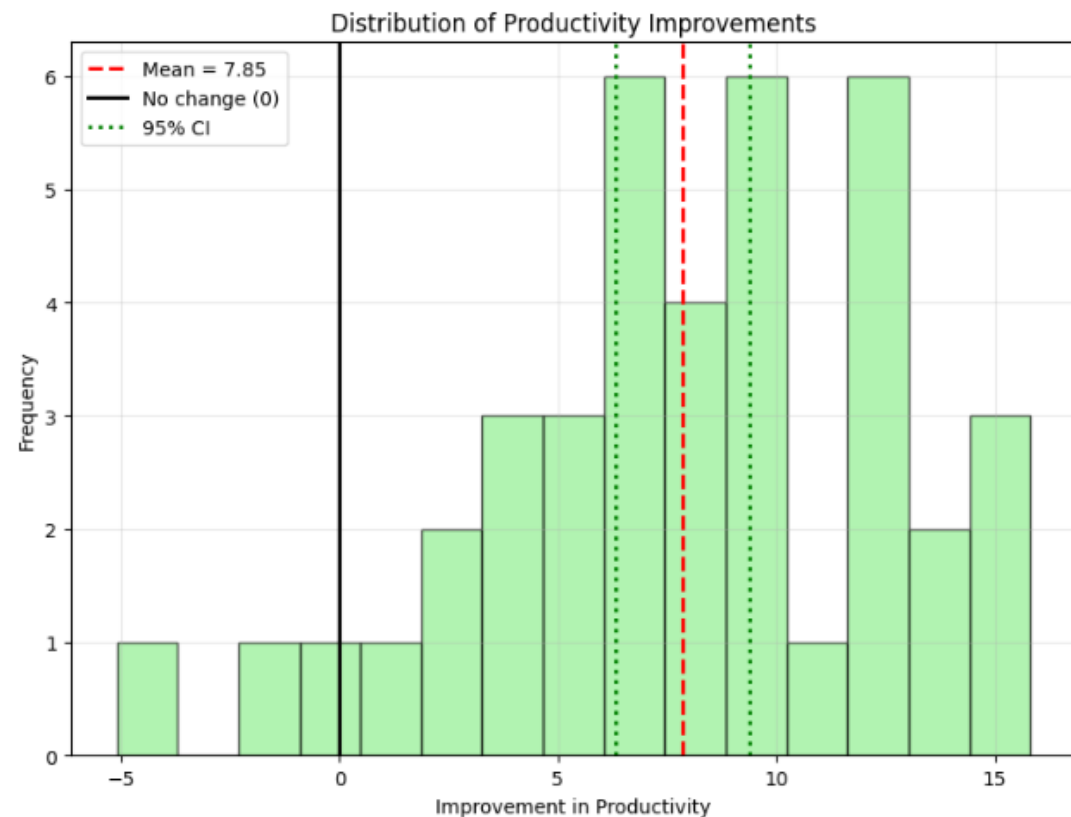
```
axes[0, 1].hist(improvement, bins=15,
```

```
axes[0, 1].axvline(mean_improvement, color='red',
```

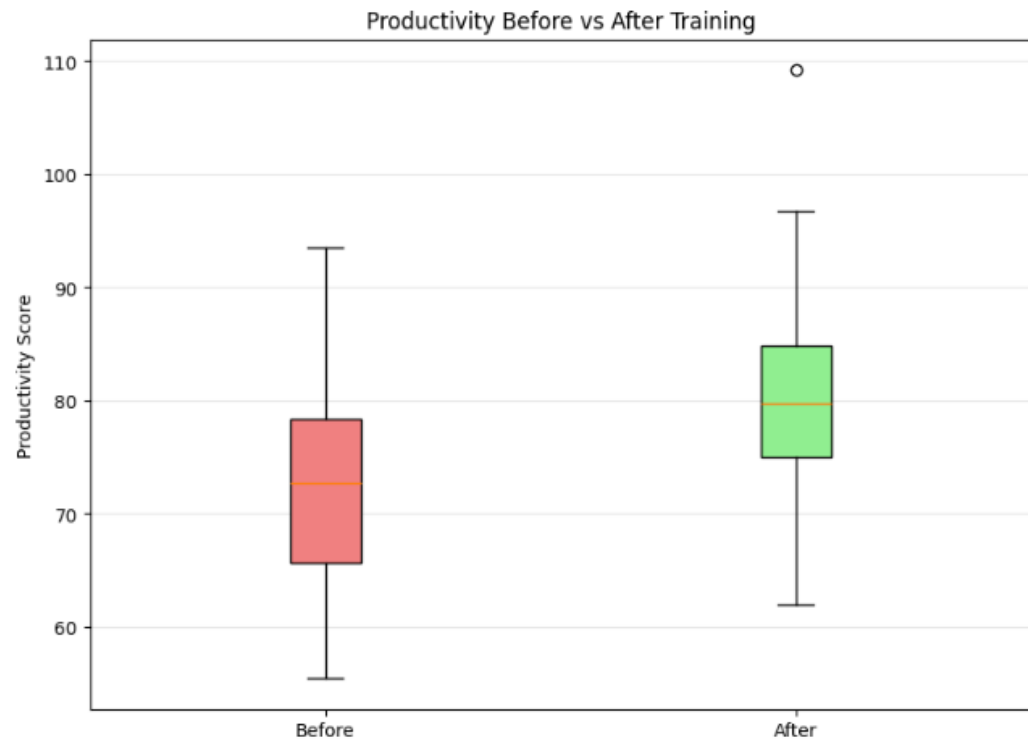
```
axes[0, 1].axvline(0, color='black',
```

```
axes[0, 1].axvline(ci_lower_p, color='green',
```

```
axes[0, 1].axvline(ci_upper_p, color='green',
```



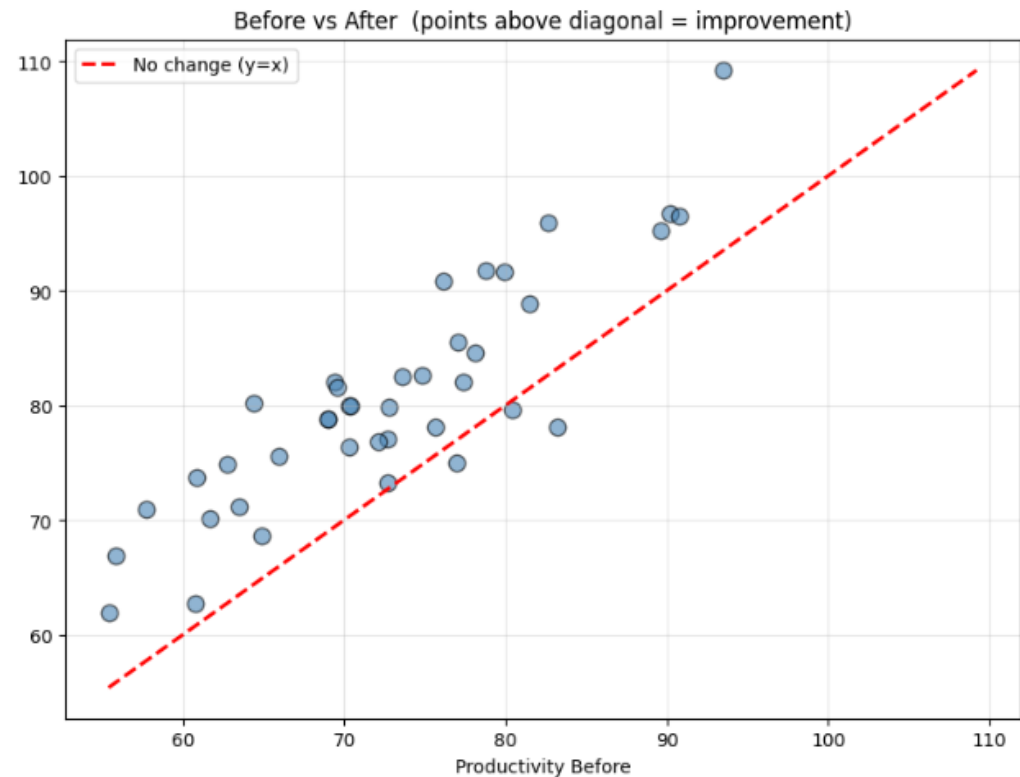
```
bp = axes[1, 0].boxplot(
    [productivity_before, productivity_after],
    tick_labels=['Before', 'After'],
    patch_artist=True,
)
```



```
axes[1, 1].scatter(productivity_before, productivity_after,
```

```
lo = min(productivity_before.min(), productivity_after.min())
hi = max(productivity_before.max(), productivity_after.max())
```

```
axes[1, 1].plot([lo, hi], [lo, hi],
```



Using Paired T-Test in ML Research Papers

The paired t-test is the **gold standard for comparing two ML models** when both are evaluated on the **same** cross-validation folds. Since both models see the same data in each fold, the fold-level scores are inherently paired — controlling for dataset difficulty.

Classification research:

Two classifiers (Model A and Model B) are evaluated on the **same 10 folds**. The per-fold accuracy difference ($\text{accuracy_B}[i] - \text{accuracy_A}[i]$) captures pure model effect, with data-split variance cancelled out.

- H_0 : $\mu_{\text{diff}} = 0$ (no systematic advantage of B over A)
- H_1 : $\mu_{\text{diff}} > 0$ (Model B is consistently better across folds)
- `ttest_rel(accuracy_B, accuracy_A)` — more powerful than two-sample test

Regression research:

Two regression algorithms are compared on the same 10 folds. The RMSE difference per fold captures which algorithm consistently performs better, independent of the fold's difficulty.

Why paired beats two-sample here: If fold 3 has particularly hard samples, both models will do worse on fold 3. The paired test controls for this, leaving only the true model difference. The two-sample test would attribute fold difficulty to model variance — making it harder to detect real differences.

ML Example — Reject H_0 : Model B Significantly Outperforms Model A on Same Folds

Two classification models are evaluated on the **exact same 10 cross-validation folds**. Since the same data partitions are used, the per-fold scores are paired observations.

- H_0 : $\mu_{\text{diff}} = 0$ (no systematic difference between models)
- H_1 : $\mu_{\text{diff}} \neq 0$ (Model B consistently differs from Model A)
- $\alpha = 0.05$

```
# Per-fold accuracy of two classifiers on the SAME 10 folds
# Because both models use the same folds, scores are paired
# Model A: current model, fold accuracies
accuracy_model_a = np.random.normal(0.82, 0.025, 10)
# Model B: new model, evaluated on the same folds (higher by ~0.06 per fold)
accuracy_model_b = accuracy_model_a + np.random.normal(0.060, 0.015, 10)
```

Paired T-Test: Same-Fold Comparison of Two Classifiers

=====:

$H_0: \mu_{\text{diff}} = 0$ (no systematic difference)

$H_1: \mu_{\text{diff}} \neq 0$ (Model B consistently differs from Model A)

```
# Paired t-test - tests if mean(model_b - model_a) == 0
t_paired_ml, p_paired_ml = ttest_rel(accuracy_model_b, accuracy_model_a)
```

```
# Per-fold differences
diff = accuracy_model_b - accuracy_model_a
mean_diff = np.mean(diff)
std_diff = np.std(diff, ddof=1)
n_folds = len(diff)
df_paired = n_folds - 1
t_crit_p = stats.t.ppf(0.975, df_paired)
ci_lo_p = mean_diff - t_crit_p * (std_diff / np.sqrt(n_folds))
ci_hi_p = mean_diff + t_crit_p * (std_diff / np.sqrt(n_folds))
```

t-statistic: 11.5532

t-critical (df=9): ± 2.2622

p-value: 0.000001

Fold-level accuracy differences (B - A):

Fold	1:	B=0.8686	A=0.8120	$\Delta=+0.0566$
Fold	2:	B=0.8304	A=0.7799	$\Delta=+0.0504$
Fold	3:	B=0.8414	A=0.7816	$\Delta=+0.0598$
Fold	4:	B=0.8814	A=0.8057	$\Delta=+0.0757$
Fold	5:	B=0.8583	A=0.8146	$\Delta=+0.0437$
Fold	6:	B=0.8533	A=0.8264	$\Delta=+0.0269$
Fold	7:	B=0.8620	A=0.8163	$\Delta=+0.0457$
Fold	8:	B=0.9428	A=0.8703	$\Delta=+0.0725$
Fold	9:	B=0.8626	A=0.8176	$\Delta=+0.0450$
Fold	10:	B=0.8957	A=0.8306	$\Delta=+0.0652$

Mean accuracy Model A: 0.8155

Mean accuracy Model B: 0.8696

Mean difference (B-A): +0.0541

95% CI for difference: [0.0435, 0.0648]

✓ REJECT H_0 ($p < 0.05$)

Model B is consistently and significantly better than Model A across all 10 evaluation folds.

The improvement CI [0.0435, 0.0648] lies entirely above 0.

→ Paper claim: Proposed model significantly outperforms baseline (paired t-test: $t(9)=11.55$, $p=0.0000$).

```
# Left: fold comparison
```

```
axes[0].plot(range(1, 11), accuracy_model_a,
```

```
axes[0].plot(range(1, 11), accuracy_model_b,
```

```
for i in range(n_folds):
```

```
    axes[0].plot([i + 1, i + 1],
```

```
                 [accuracy_model_a[i], accuracy_model_b[i]], 'gray',
```

```
# Right: distribution of differences
```

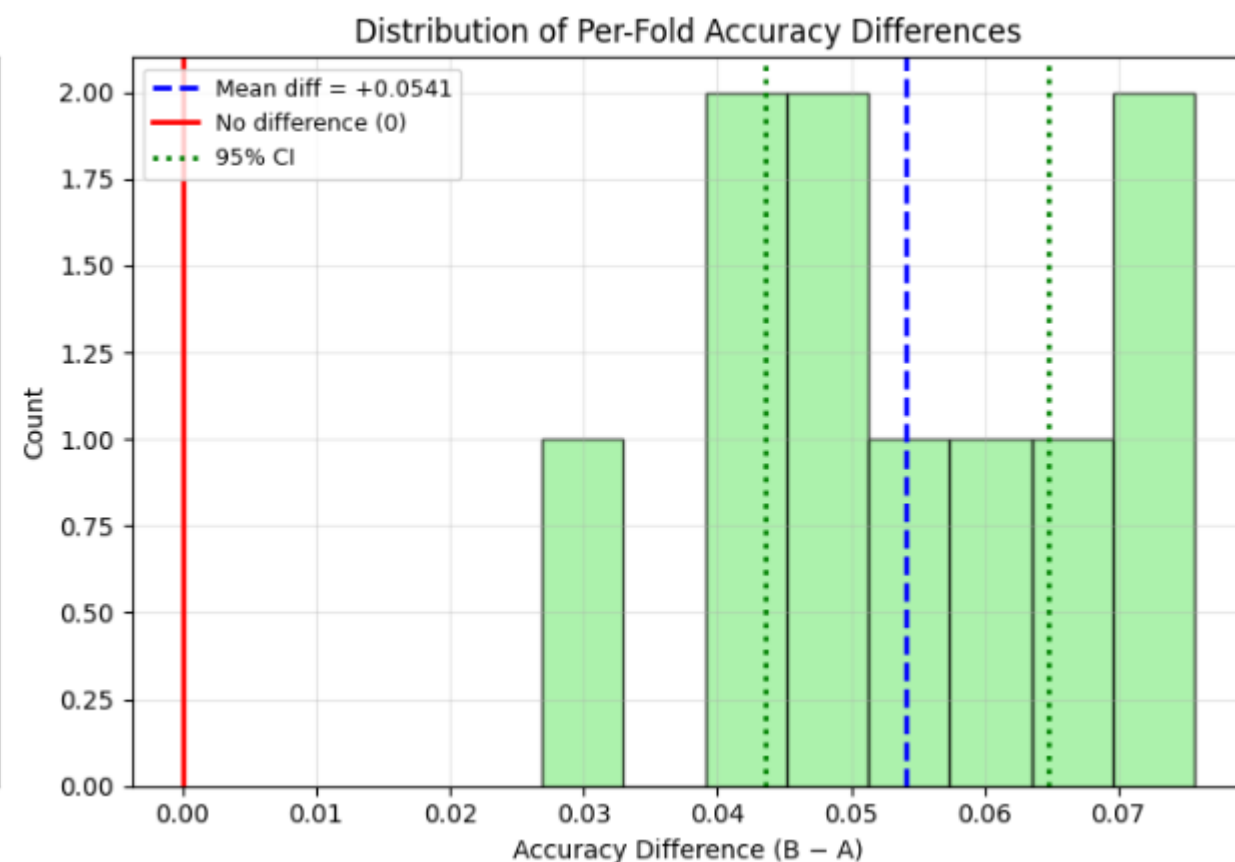
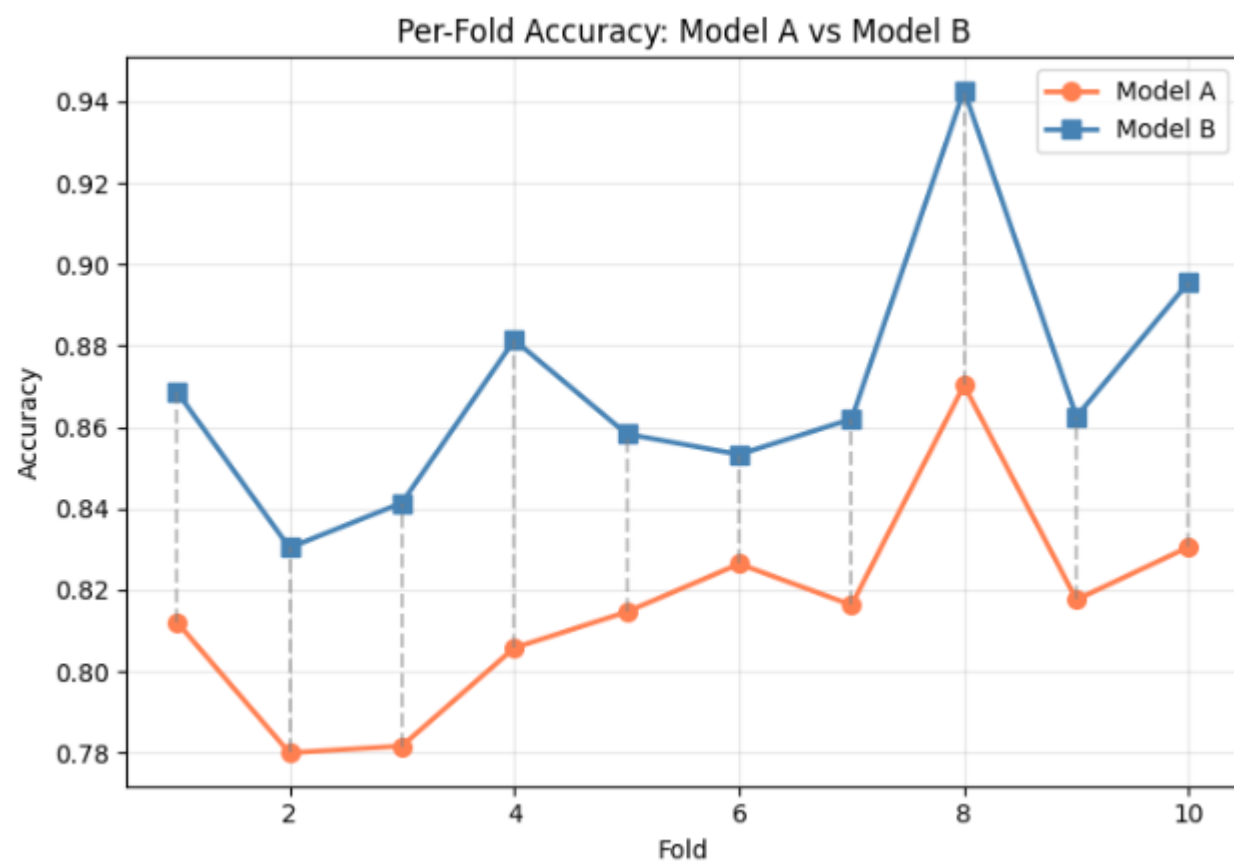
```
axes[1].hist(diff, bins=8,
```

```
axes[1].axvline(mean_diff, color='blue',
```

```
axes[1].axvline(0, color='red',
```

```
axes[1].axvline(ci_lo_p, color='green',
```

```
axes[1].axvline(ci_hi_p, color='green',
```



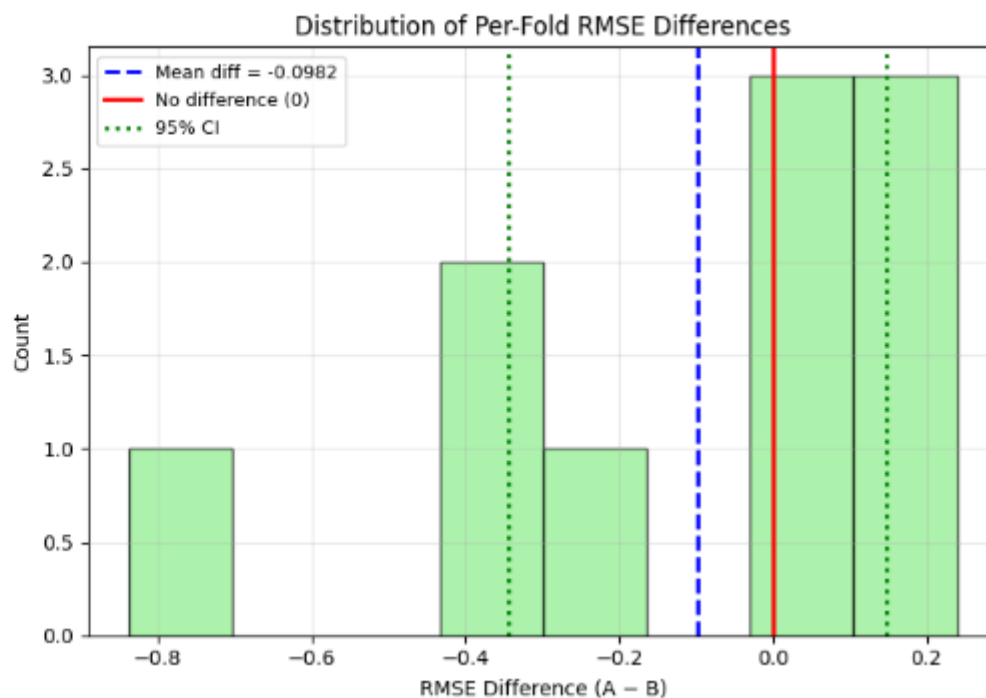
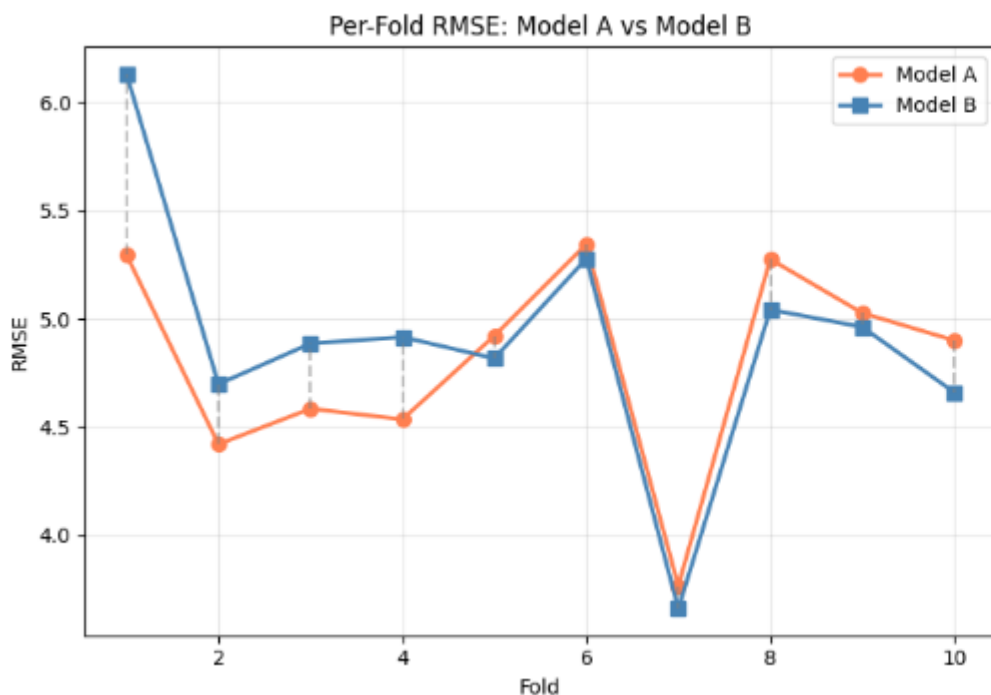
ML Example — Fail to Reject H_0 : No Consistent Improvement

Two regression models are evaluated on the same 10 folds. Model B uses a slightly more complex architecture. The improvements are inconsistent across folds.

- H_0 : $\mu_{\text{diff}} = 0$ (no systematic RMSE reduction)
- H_1 : $\mu_{\text{diff}} \neq 0$

✗ FAIL TO REJECT H_0 ($p = 0.3888 \geq 0.05$)

Model B shows no consistent RMSE improvement over Model A.
Differences vary across folds – no systematic advantage.



8. Chi-Square Tests

Chi-Square tests work with **categorical data** — counts and frequencies. They answer questions like "is the observed distribution what we expected?" or "are two categorical variables related?"

The chi-square statistic measures how far observed counts deviate from expected counts:

$$\chi^2 = \sum [(O - E)^2 / E]$$

- O = Observed frequency
- E = Expected frequency
- Sum is over all categories/cells

- χ^2 is always **non-negative** (≥ 0).
- $\chi^2 = 0$ means perfect match between observed and expected.
- Larger $\chi^2 \rightarrow$ more evidence against H_0 .
- The χ^2 distribution is **right-skewed** (one-tailed rejection region on the right).

8.1 Chi-Square Goodness of Fit Test

Purpose: Tests whether a single categorical variable follows an expected/hypothesized distribution.

Real-World Case: Website Traffic Distribution

A website expects **equal traffic** across all days of the week. We test if observed traffic matches this expectation.

- **H₀:** Traffic is equally distributed across all days.
- **H₁:** Traffic is NOT equally distributed.
- **$\alpha = 0.05$, $df = k - 1 = 6$** (k = number of categories = 7 days)

If **$\chi^2_{\text{statistic}} > \chi^2_{\text{critical}}$** → Reject H₀.

Chi-Square Goodness of Fit: Weekly Traffic Distribution

=====

H₀: Traffic is equally distributed across all days

H₁: Traffic is NOT equally distributed

```
days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
```

```
base_mean = 170
```

```
base_std = 15
```

```
# Weekday traffic (Mon-Fri)
```

```
weekday_traffic = np.random.normal(base_mean, base_std, 5).astype(int)
```

```
# Weekend traffic higher (Sat-Sun)
```

```
weekend_traffic = np.random.normal(base_mean + 40, base_std, 2).astype(int)
```

```
observed = np.concatenate([weekday_traffic, weekend_traffic])
```

```
total_visitors = observed.sum()
```

```
# — Step 2: Expected frequencies (equal distribution) —
```

```
expected = np.array([total_visitors / 7] * 7)
```

```
# — Step 3: Chi-square goodness of fit test —
```

```
chi2_stat, p_value = chisquare(observed, expected)
```

```
df_chi = len(days) - 1
```

```
critical_v = stats.chi2.ppf(0.95, df_chi)
```

Day	Observed	Expected	Diff
Mon	177	188.57	-11.57
Tue	167	188.57	-21.57
Wed	179	188.57	-9.57
Thu	192	188.57	+3.43
Fri	166	188.57	-22.57
Sat	206	188.57	+17.43
Sun	233	188.57	+44.43

Test Results:

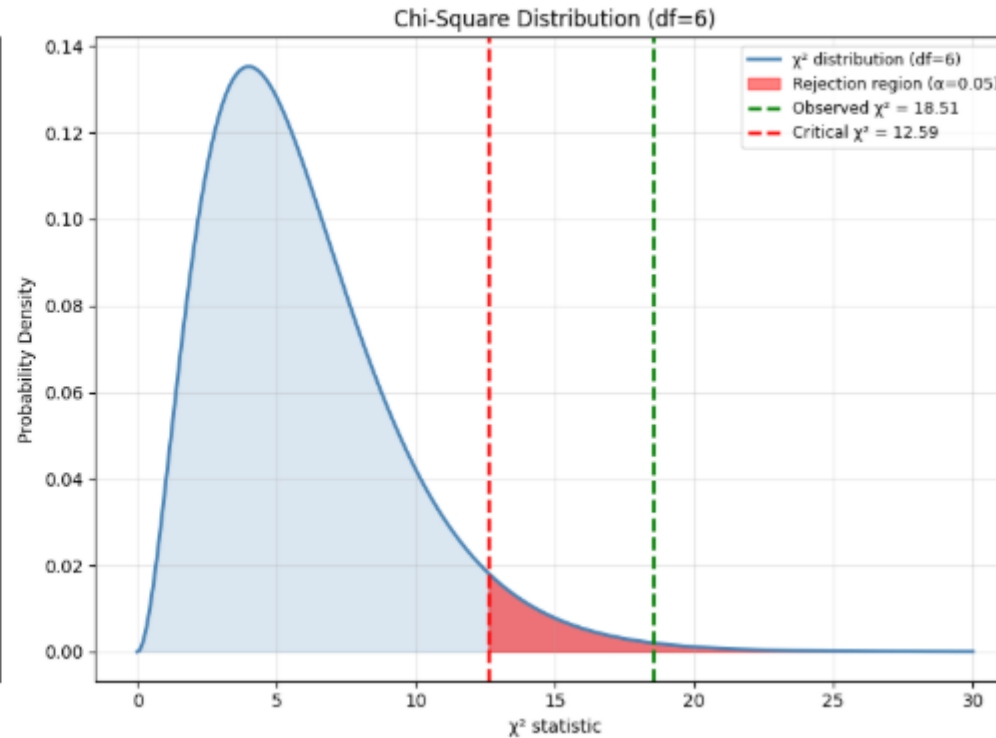
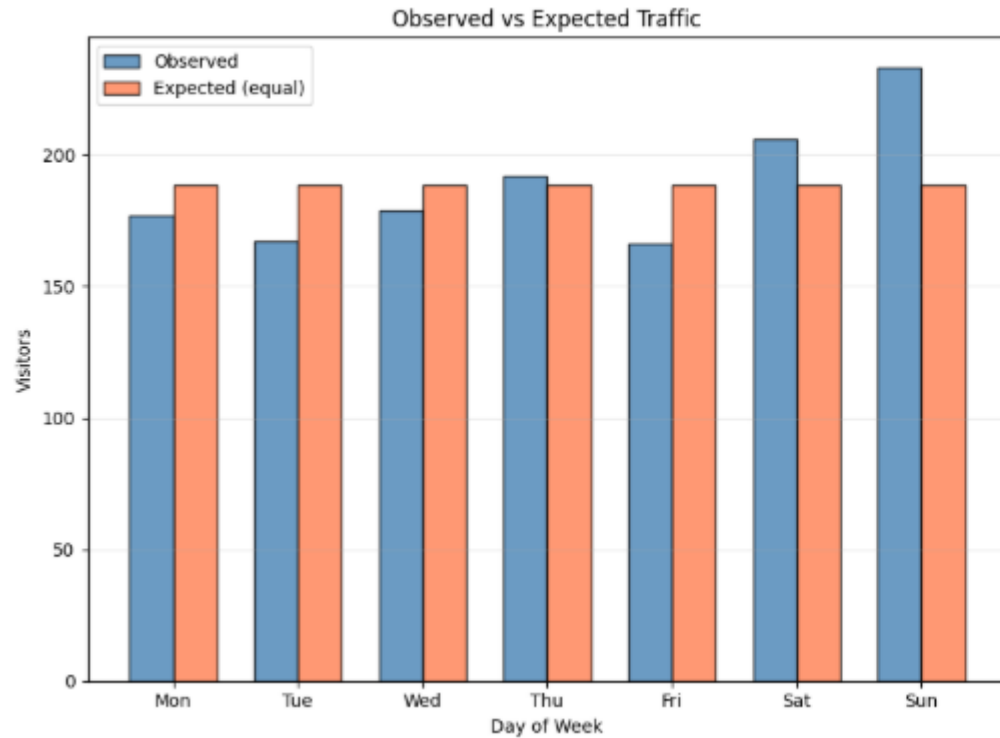
χ^2 statistic: 18.5061

Critical value (df=6, $\alpha=0.05$): 12.59

Degrees of freedom: 6

p-value: 0.0051

✓ REJECT H₀ (p < 0.05): Traffic is NOT equally distributed
Business insight: Weekend traffic is significantly higher
Recommendation: Allocate more server resources for weekends



- `stats.chi2.ppf(0.95, df)` – finds χ^2_{critical} . Note: unlike t-tests (two-tailed), chi-square tests are **always one-tailed** (right-tail only), so we use `0.95` not `0.975`.

8.2 Chi-Square Test of Independence

Purpose: Tests whether **two categorical variables are associated** with each other, or if they are independent.

Real-World Case: Customer Churn vs Service Plan

A telecom company wants to know: *Is customer churn associated with the type of service plan (Basic, Standard, Premium)?*

- **H₀:** Churn is **independent** of service plan (no association).
- **H₁:** Churn is **associated** with service plan.
- **df** = (rows - 1) × (columns - 1) = (2-1) × (3-1) = 2

Using Chi-Square Test of Independence in ML Research Papers

Feature selection for classification:

Before building a classifier, you want to know which categorical features are actually related to the target label. Running a chi-square test of independence between each categorical feature and the target (e.g., spam/not spam, churn/retained) tells you which features carry predictive information.

- H_0 : Feature X is independent of the target label (no predictive value).
- H_1 : Feature X is associated with the target (potentially useful feature).

Model fairness analysis:

After training a classifier, you test whether the model's **predictions are independent of a protected attribute** (e.g., gender, race). If they are associated, the model may be discriminatory.

- H_0 : Model predictions are independent of the protected attribute.
- H_1 : Predictions are associated with the protected attribute (potential bias).

```
# — Contingency table —————
data = np.array([[150, 80, 50], [50, 120, 150]])
row_labels = ['Churned', 'Retained']
col_labels = ['Basic', 'Standard', 'Premium']

contingency_df = pd.DataFrame(data, index=row_labels, columns=col_labels)
print("Observed Frequencies:")
print(contingency_df)
print(f"\nTotal sample size: {data.sum()}")
```

```
# — Chi-square test of independence —————
```

```
chi2, p_value, dof, expected = stats.chi2_contingency(data)
```

```
print(f"\nChi-square statistic: {chi2:.4f}")
print(f"P-value: {p_value:.6f}")
print(f"Degrees of freedom: {dof}")
```

```
critical_chi2 = stats.chi2.ppf(0.95, dof)
```

```
print(f" $\chi^2$ _critical ( $\alpha=0.05$ ,  $df={dof}$ ): {critical_chi2:.2f}")
```

```
expected_df = pd.DataFrame(expected, index=row_labels, columns=col_labels)
print("\nExpected Frequencies (under independence):")
print(expected_df.round(2))
```

Observed Contingency Table:

	Basic	Standard	Premium	Total
Churned	150	80	50	280
Retained	350	420	450	1220
Total	500	500	500	1500

All Tests Covered in This Notebook

Test	Data Type	Groups	Python Function	Key Output
One-Sample t-test	Continuous	1 sample vs known value	<code>ttest_1samp(a, μ_0)</code>	t, p-value
Two-Sample t-test	Continuous	2 independent groups	<code>ttest_ind(a, b)</code>	t, p-value
Paired t-test	Continuous	2 paired groups (before/after)	<code>ttest_rel(a, b)</code>	t, p-value
Chi-Square GoF	Categorical	1 variable vs expected dist.	<code>chisquare(obs, exp)</code>	χ^2 , p-value
Chi-Square Independence	Categorical	2 categorical variables	<code>chi2_contingency(table)</code>	χ^2 , p-value, df